

Práctica 2

Nombre proyecto

Turbina Tour And Resort

Asignatura

Programación Orientada a Objetos

Grupo

01

Equipo

05

Integrantes

Jhoneyker Delgado Urbina
Emmanuel Valencia Lopera
Simón Guarín Cortés
Jesus Camilo Miranda Aguirre
Andrés Jacobo Leal Aguirre

Docente

Jaime Alberto Guzmán Luna

Universidad Nacional de Colombia- Sede Medellín

Fecha

Febrero 2025

1. Descripción general de la solución.

El objetivo principal de *Turbina Tour and Resort* es implementar un sistema integral de reservas de vuelos con o sin alojamiento, diseñado para optimizar las operaciones de consulta, compra y gestión de vuelos y alojamientos, brindando una experiencia eficiente y accesible para los administradores. Este sistema busca resolver las dificultades asociadas con la venta de tiquetes, la gestión de aerolíneas y el manejo de alojamientos en una zona aeroportuaria. A través de una interfaz intuitiva y orientada al administrador, se ofrecerán herramientas prácticas que simplifican y agilizan la administración de estas tareas. El programa está diseñado para satisfacer las necesidades específicas establecidas en los requisitos funcionales, garantizando una solución efectiva y acorde a las exigencias del entorno aeroportuario.

Requisitos Funcionales

Búsqueda de Vuelos (Esencial)

1. Permitir buscar vuelos según los siguientes criterios:
 - Por destino.
 - Por destino y fecha.
 - Por ID de vuelo.

Gestión de Aerolíneas (Esencial)

2. Permitir la creación de aerolíneas especificando:
 - Nombre de la aerolínea.
3. Permitir la eliminación de aerolíneas especificando:
 - Nombre de la aerolínea.

Gestión de Vuelos (Esencial)

4. Permitir la creación de vuelos asociados a una aerolínea con los siguientes detalles:
 - Origen.
 - Destino.
 - Distancia.
 - Fecha de salida.
 - Hora de salida.
5. Permitir la cancelación de vuelos asociados a una aerolínea con los siguientes detalles:
 - Nombre de la aerolínea
 - ID del vuelo

Gestión de Alojamientos (Esencial)

6. Permitir la creación de alojamientos especificando:
 - Nombre propio del alojamiento.
 - Ubicación.
 - Precio por día.

- Número de estrellas.
- 7. Permitir la eliminación de alojamientos especificando:
 - Nombre del alojamiento

Compra de Tiquetes (Esencial)

- 8. Permitir comprar un tiquete a través de los siguientes pasos:
 - Buscar vuelos por destino o por destino y fecha.
 - Seleccionar la aerolínea y el ID del vuelo.
 - Agregar la información del pasajero.
 - Nombre del pasajero.
 - Edad.
 - Número de pasaporte.
 - Correo electrónico.
 - Clase de la silla (económica o ejecutiva).
 - Ubicación de la silla.
- 9. Agregar Alojamientos al Tiquete:
 - Una vez comprado el tiquete, permitir añadir una reserva de alojamiento, especificando:
 - Destino del vuelo.
 - Tiempo de estadía.

Modificación de Tiquetes (Esencial)

- 10. Permitir realizar modificaciones a un tiquete ya comprado:
 - Cambiar la silla seleccionada.
 - ID del tiquete.
 - Ubicación de la silla.
 - Clase de la silla (económica o ejecutiva).
 - Cambiar el alojamiento (si el tiquete ya tiene uno asociado).
 - ID del tiquete.
 - Nombre del alojamiento .
 - Tiempo de estadía.

Requisitos no funcionales

- 1. Eficiente y accesible.
 - Fácil de usar: por la guía que se le hace al usuario a través de los numerales para cumplir su necesidad dentro del programa.
 - Rápido y óptimo :No sabemos con exactitud cuán rápido es el sistema pero al ser un programa en un ámbito de práctica el tiempo de respuesta es óptimo y eficiente ya que en sus funcionalidades no requiere de procesos largos.
- 2. Confiable
 - Proteger datos : El encapsulamiento que se le ha realizado al programa hace que los datos no sean accedidos y modificados desde otra funcionalidad u objeto del programa.
 - Reducir errores : El programa siempre sigue una secuencia de pasos para que sea fácil de entender para quien utiliza la aplicación por lo que se reducen los

errores de uso de la aplicación y siempre que hay un error se le expresara al usuario de manera adecuada al usuario que esté utilizando el programa de TURBINA TOUR AND RESORT.

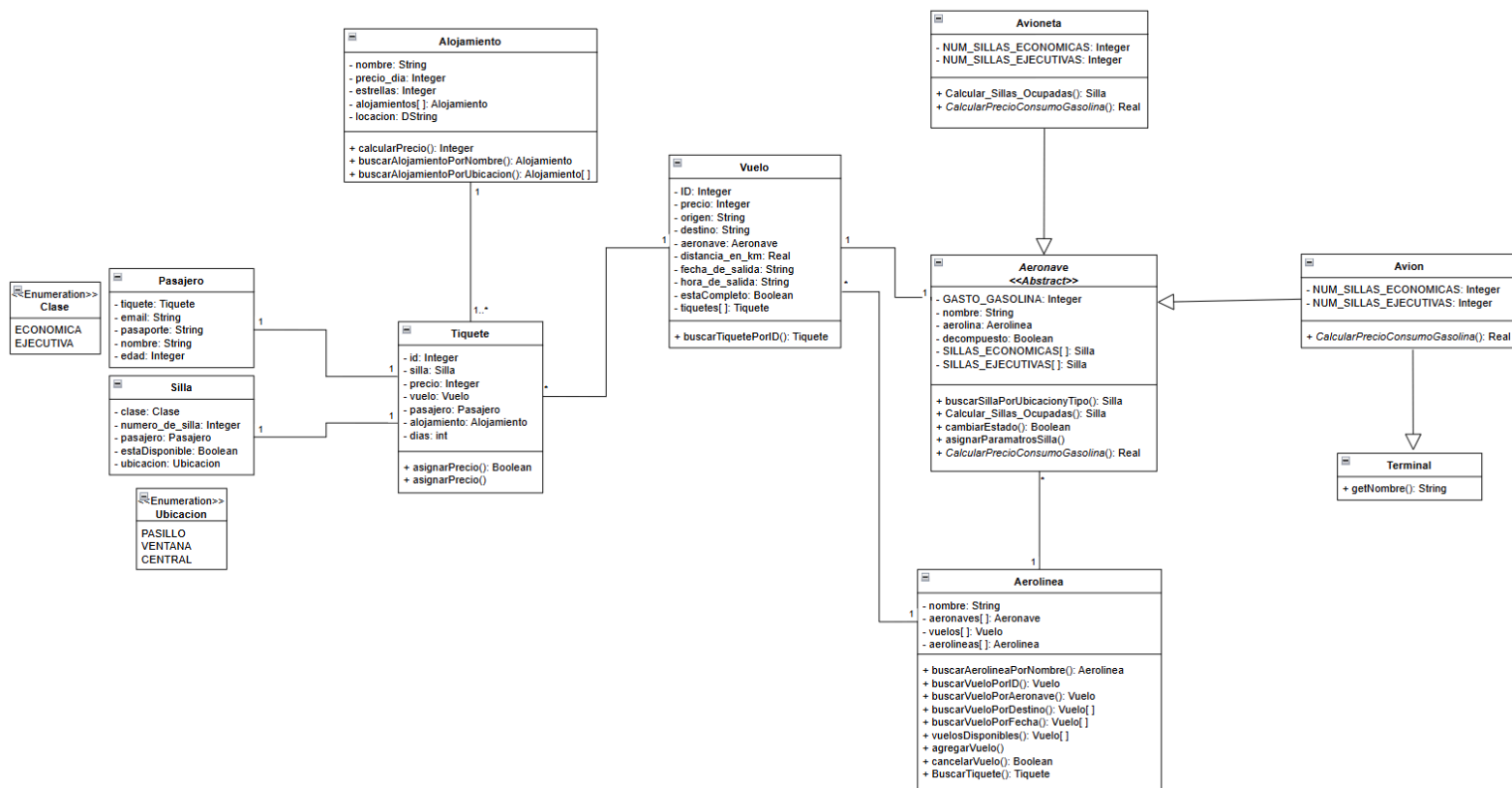
3. Usabilidad

- Fácil uso para el usuario final: Todo está especificado en la interfaz para que sea fácil al usuario que necesidad desea cumplir con la aplicación. La especificación está dada por medio de numerales, descripción de cada funcionalidad y precisión en los requerimientos que debe dar el usuario.

4. Mantenibilidad

- Adaptable a actualizaciones o correcciones. El programa fue diseñado para poder hacer más funcionalidades e implementaciones según las necesidades y requerimientos de este sector del transporte aéreo. El código cumple con las características de la POO por lo que la reutilización de código se hace esencial.
- Documentación técnica adecuada: Está propia memoria escrita que se está presentando hace parte de la documentación técnica ,que especifica cómo funciona el programa, que soluciona y como se realizan las implementaciones

2. Descripción del diseño estático del sistema en la especificación UML.



[Diagrama sin título.drawio - draw.io](https://drawio.io)

Clase Aerolínea

Esta clase tiene el rol de almacenar las aerolíneas con sus respectivos vuelos y aeronaves y la información de cada uno de estos, además se agregan nuevos vuelos a las aerolíneas o se cancelan y se tiene la posibilidad de hacer una búsqueda de los vuelos (dependiendo de la Aerolínea) y búsqueda según el nombre de la Aerolínea.

Clase Pasajero

Esta clase tiene el rol de asignar un Pasajero a un Tiquete junto con los datos de este, tales como: el nombre, edad, tiquete, email, pasaporte.

Clase Tiquete

Esta clase tiene el rol de crear un tiquete para un respectivo Pasajero y contener un alojamiento (En caso de que el pasajero lo requiera). También tiene el trabajo de asignar valor a cada tiquete.

Clase Vuelo

Esta clase tiene el rol de crear los vuelos junto con una lista de tiquetes asociada a cada vuelo, además de también asignársele a cada uno de estos una aeronave. También, se puede hacer la búsqueda de un tiquete según el ID en la lista de tiquetes respectiva del vuelo.

Clase Alojamiento

Esta clase tiene el rol de ser el molde para crear alojamientos, además se almacenan los alojamientos. Aquí se calcula el precio por cada alojamiento y también se buscan alojamientos existentes según la ubicación o por nombre.

Clase Aeronave

Esta clase es abstracta, tiene el rol de definir los datos principales de cualquier tipo de aeronave. Además se realizan procesos para saber si el vuelo que realiza esta aeronave ya está completo, también se calculan que cantidad de sillas ocupadas y se actualiza la ocupación de los puestos.

Clase Avión

Esta clase tiene el rol de ser molde para crear aeronaves de tipo avión, se tienen establecidas unas capacidades de sillas ejecutivas y económicas. Al heredar de la clase Aeronave aplica todo lo mencionado anteriormente. Se calcula el precio según el gasto de gasolina para que en otro proceso fuera de la clase se sume al precio del tiquete.

Clase Avioneta

Esta clase tiene el rol de ser molde para crear aeronaves de tipo avioneta, se tienen establecidas unas capacidades de sillas ejecutivas y económicas. Al heredar de la clase Aeronave aplica todo lo mencionado anteriormente. Se calcula el precio según el gasto de gasolina para que en otro proceso fuera de la clase se sume al precio del tiquete y además se calcula de otra manera la cantidad de sillas ocupadas.

Clase Enum "Clase"

Esta clase tiene el rol de almacenar los tipos de clase que tiene un aeronave y en los cuales pueden escoger los pasajeros. En este caso están los enumerados: Ejecutiva y económica.

Clase Silla

La clase Silla modela las características de una silla dentro de una aeronave. Almacena información como el número de silla, su clase (económica o ejecutiva), el pasajero asignado, su disponibilidad y su ubicación.

Clase Enum “Ubicación”

Esta clase tiene el rol de almacenar los tipos de ubicación que tiene una silla de una aeronave y en los cuales pueden escoger los pasajeros. En este caso están los enumerados: Pasillo, ventana y central.

Clase Terminal

Encargado de hacer la diferenciación entre un Avión y Avioneta

Relaciones

Aerolínea — Vuelo

Una aerolínea tiene 0 o muchos vuelos.

Aerolínea — *Aeronave*

Una aerolínea tiene 0 o muchas aeronaves.

Vuelo — Tiquete

Un vuelo tiene muchos tiquetes.

Vuelo — *Aeronave*

Un vuelo tiene una sola aeronave.

Tiquete — Alojamiento

Uno o muchos tiquetes tienen un solo alojamiento.

Tiquete — Pasajero

Un tiquete se relaciona con solo un pasajero.

Tiquete — Silla

Un tiquete se relaciona con una sola silla.

Avion—> *Aeronave*

Avión hereda de Aeronave.

Avion—> *Terminal*

Avión hereda de Terminal.

Avioneta —> *Aeronave*

Avioneta hereda de Aeronave.

3. Descripción de la implementación de características de programación orientada a objetos en el proyecto.

Clases Abstractas

```
from abc import ABC, abstractmethod

class Aeronave(ABC):

    # CONTRUCTOR
    def __init__(self, nombre, aerolinea=0):
        self.Gasto_gasolina = 10000
        self._descompuesto = False
        self._SILLAS_ECONOMICAS = []
        self._SILLAS_EJECUTIVAS = []
        self._nombre = nombre
        if aerolinea==0:
            self._aerolinea = None
        else:
            self._aerolinea = aerolinea
```

Package gestorAplicación.hangar, clase Aeronave.

Esta clase abstracta permite la definición de una plantilla para las subclases(Avión y Avioneta), permitiendo así la reutilización del código y flexibilidad al momento de definir el método según el contexto de la subclase.

Métodos Abstractos

```
@abstractmethod
def Calcular_Consumo_Gasolina(self, distancia_en_km):
```

Package gestorAplicación.hangar, clase Aeronave, Línea 172

Este método abstracto que pertenece a la clase abstracta Aeronave indica que debe de ser utilizado por las subclases que hereden de ella. Este método se definirá en las subclases para calcular el consumo de gasolina según la distancia del vuelo y su autonomía.

Herencia

```
class Avion(Aeronave):  
    _NUM_SILLAS_ECONOMICAS = 24  
    _NUM_SILLAS_EJECUTIVAS = 12
```

Package gestorAplicación.hangar.clase Avión.

La herencia es aplicada en la clase Avion con la super clase Aeronave de la cual hereda el nombre de la aeronave, la aerolínea en la cual esta registrada, si la aeronave está descompuesta, y los atributos de sillas economicas y ejecutivas, que más tardes serán sobrescritos para el contexto de la clase, además de definir el método abstracto calcularPrecioConsumoGasolina declarado en Aeronave (Aeronave es una clase abstracta), dando orden a las instrucciones y reutilizando código.

Ligadura dinámica

```
def getNombre(self):  
    texto = super().getNombre() + "_A"  
    return texto
```

Package gestorAplicación.hangar.clase Terminal.

En el segundo método estamos sobrescribiendo el método heredado de la clase padre (Aeronave)..El método sobrescrito retorna el nombre del objeto (almacenado en el atributo nombre), concatenado con un sufijo(_A). Para acceder al atributo nombre, utilizamos el método get nombre, que es el **getter** correspondiente.

```
def Calcular_Sillas_Ocupadas(self):  
    cont = 0  
    for i in self.getSILLASECONOMICAS():  
        if i.isEstado():  
            cont += 1  
    for j in self.getSILLASEJECUTIVAS():  
        if j.isEstado():  
            cont += 1  
    return "Sillas ocupadas en la avioneta"+cont
```


Package gestorAplicación.hangar clase Avioneta línea 31.

En el segundo método estamos sobrescribiendo el método heredado de la clase padre (Aeronave). El método sobrescrito recorrerá los arreglos de sillas ejecutivas y económicas de la avioneta, luego verifica la cantidad que están ocupadas. Posteriormente, retorna un mensaje con la cantidad total de sillas ocupadas.

Atributos de clase

```
_aerolineas = []
```

Package gestorAplicación.adminVuelos clase Aerolínea línea 13.

Esto indica que cada instancia de la clase Aerolínea tendrá el atributo común aerolíneas que para este caso es una lista, según la lógica del programa cada vez que se cree una instancia de esta clase se añadirá a la lista común dando un mejor orden y un mejor manejo de los atributos.

```
_alojamientos = []
```

Package gestorAplicación.alojamiento clase Alojamiento línea 5.

Esto indica que cada instancia de la clase Alojamiento tendrá el atributo común alojamientos que para este caso es una lista, según la lógica del programa cada vez que se cree una instancia de esta clase se añadirá a la lista común dando un mejor orden y un mejor manejo de los atributos.

Método de clase

```
@staticmethod
def buscarAerolineaPorNombre(nombre2):
    retorno = None
    i = 0
    while i < len(Aerolinea.getAerolineas()):
        if Aerolinea.getAerolineas()[i].getNombre().lower() == nombre2.lower():
            # SI ENCUENTRA UNA AEROLINEA CUYO NOMBRE COINCIDA CON EL PARAMETRO
            # RETORNA ESA AEROLINEA.
            retorno = Aerolinea.getAerolineas()[i]
        i += 1
    return retorno
```

Package gestorAplicación.adminVuelos clase Aerolinea línea 33.

Método de clase que tiene como finalidad recibir un nombre de aerolínea y buscar entre las aerolíneas disponibles si hay alguna con ese nombre, si es así retorna ese objeto aerolínea, si no, retorna none.

Uso de constantes

```
self.GASTO_GASOLINA = 10000
```

Package gestorAplicacion.hangar clase Aeronave línea 9.

El uso de constantes indica que este será un atributo que no podrá ser cambiado más adelante en el programa; el valor definido para este caso es un entero (10000) y su nivel de accesibilidad será private.

Manejo de referencia self

```
Aerolinea._aerolineas.append(self)
```

Package gestorAplicacion.adminvuelos clase Aerolínea línea 24.

Utilizamos self para hacer referencia al objeto creado y añadirlo a la lista de aerolíneas.

```
ticket.setPasajero(self)
```

Package gestor Aplicación.adminVuelos clase Pasajero línea 11.

En este caso el propósito de self es pasar la instancia actual del objeto pasajero como argumento al método set pasajero del objeto ticket.

Caso de enumeración

```
from enum import Enum

class Clase(Enum):
    ECONOMICA = 10000
    EJECUTIVA = 70000
```

Package gestorAplicacion.hangar clase Clase

Este enum permite predefinir el tipo de sillitas de la aeronave y asocia un valor para cada tipo de silla y es de gran importancia porque la información contenida en este no podrá ser modificada durante la ejecución además de facilitar la obtención del precio de las sillitas.

Herencia Múltiple

```
class Avion(Terminal,Aeronave):  
    _NUM_SILLAS_ECONOMICAS = 24  
    _NUM_SILLAS_EJECUTIVAS = 12
```

Package gestorAplicación.hangar.clase Avion línea

La clase Avión implementa el concepto de herencia múltiple dentro de la Programación Orientada a Objetos (POO). Esta clase hereda de Terminal y Aeronave. De Terminal: Obtiene el método getNombre()y de Aeronave(clase abstracta) Hereda atributos y métodos específicos de la aeronave y un método abstracto, que Avión debe implementar obligatoriamente.

Sobrecarga de “constructores”

```
#CONSTRUCTOR  
@multimethod  
def __init__(self, id:int, precio:int, origen:str, destino:str, aeronave: Aeronave, distancia:int, fecha_de_salida:str, hora_de_salida:s  
    self._ID = id  
    self._precio = precio  
    self._origen = origen  
    self._destino = destino  
    self._aeronave = aeronave  
    self._distancia_en_km = distancia  
    self._fecha_de_salida = fecha_de_salida  
    self.setHora_de_salida(hora_de_salida)  
    self.getAeronave().getAerolínea().agregarVuelo(self)  
    self._tiquetes = []  
    self._estaCompleto = False  
  
@multimethod  
def __init__(self, id:int, precio:int, origen:str, destino:str, aeronave: Aeronave, distancia:int, fecha_de_salida:str):  
    self._ID = id  
    self._precio = precio  
    self._origen = origen  
    self._destino = destino  
    self._aeronave = aeronave  
    self._distancia_en_km = distancia  
    self._fecha_de_salida = fecha_de_salida  
    self.getAeronave().getAerolínea().agregarVuelo(self)  
    self._tiquetes = []  
    self._estaCompleto = False
```

package gestor Aplicación.admin Vuelos.Vuelo línea 14

El primer constructor recibe como parámetros un ID, un precio, un origen, un destino, una aeronave, una distancia, una fecha de salida y una hora de salida, respectivamente. Este constructor también incluye el método setHora_de_salida(), que permite asignar la hora de salida. Además, obtiene la aerolínea correspondiente y agrega el vuelo a su lista de vuelos.El segundo constructor recibe la misma información que el primero, excepto la hora de salida.La sobrecarga de constructores en este caso permite crear objetos de tipo Vuelo de distintas maneras, dependiendo de la cantidad de información disponible en el momento de la instanciación. Esto brinda mayor flexibilidad al momento de registrar vuelos en el sistema.

Sobrecarga de métodos

```
def __init__(self, nombre:str, vuelos:Vuelo):
    self._aeronaves = []
    self._vuelos = vuelos
    self._nombre = nombre
    Aerolinea._aerolineas.append(self)
@multimethod
def __init__(self, nombre:str):
    self._aeronaves = []
    self._nombre = nombre
    self._vuelos=[]
    Aerolinea._aerolineas.append(self)
```

package gestor Aplicación.admin Vuelos.Aerolinea línea 16

El primer método recibe como parámetros un **nombre** y un **vuelo**, permitiendo inicializar la aerolínea con un vuelo desde su creación. El segundo método recibe únicamente un **nombre**, creando la aerolínea sin vuelos iniciales. Ambos métodos agregan las aerolíneas creadas a una lista global de aerolíneas. La sobrecarga del método `__init__` permite dos formas distintas de instanciar la clase Aerolínea, según la información disponible al momento de la creación del objeto.

4. Descripción de cada una de las 5 funcionalidades implementadas.

Funcionalidad #1 :Ver todos los vuelos disponibles por aerolínea

Descripción

Esta funcionalidad le permitirá al usuario ver todos los vuelos disponibles operados por cada una de las diferentes aerolíneas. Se realizará una búsqueda global, sin especificación de fecha ni horario de todos los vuelos disponibles. El sistema validará los vuelos por medio de una secuencia de pasos excluyendo aquellos que se encuentren llenos para no almacenarlos en la lista de vuelos disponibles. Finalmente el sistema mostrará la información principal de cada uno de ellos organizada en una tabla, con información como:

ID	Precio	Origen	Destino	Fecha	Hora de salida	Nombre de la aeronave
----	--------	--------	---------	-------	----------------	-----------------------

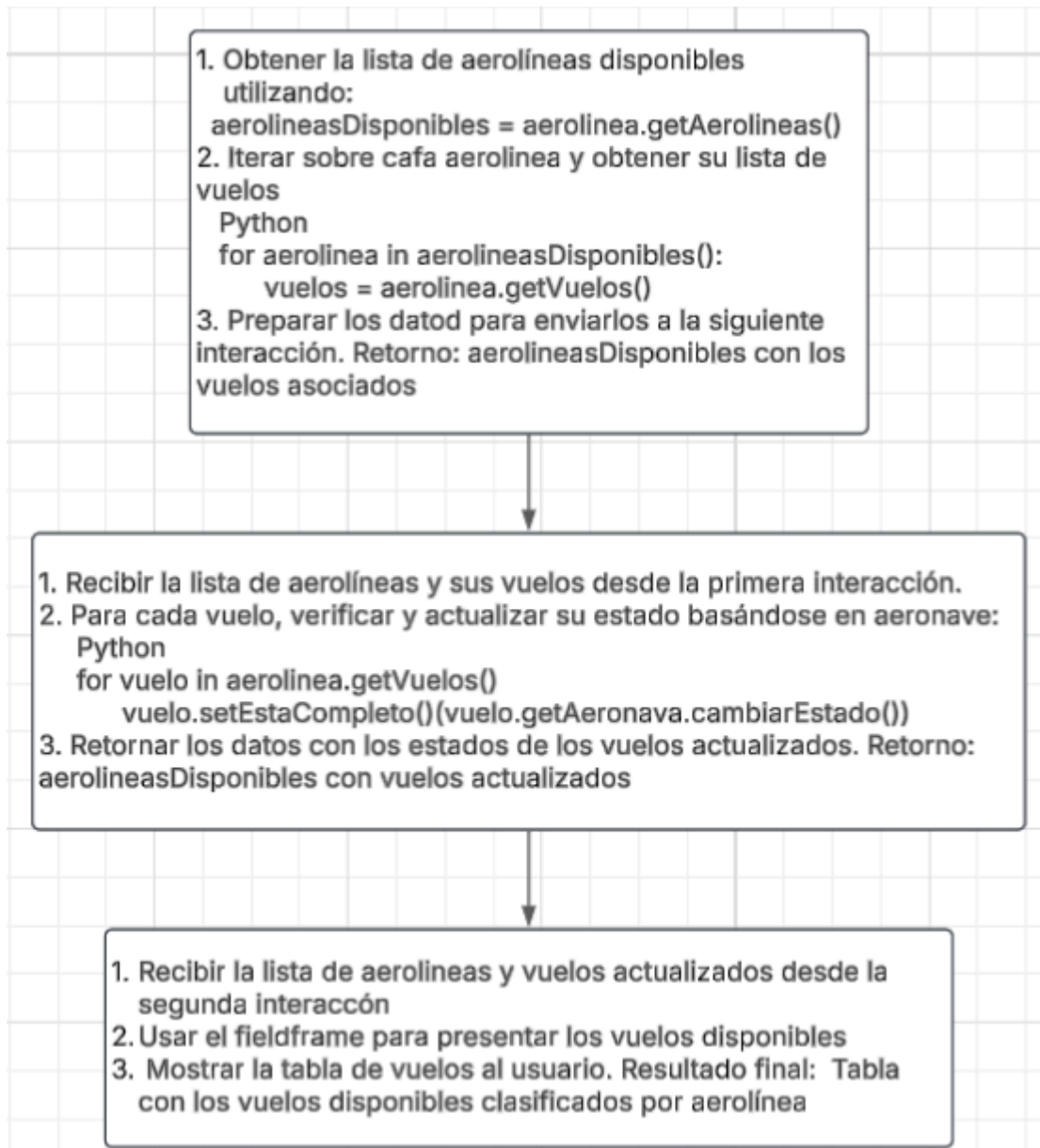
Cada una de esas columnas haciendo referencia a:

- **ID del vuelo:** Identificador único que permite distinguir cada vuelo.
- **Precio:** Costo del boleto correspondiente al vuelo.
- **Origen:** Ciudad desde donde parte el vuelo.
- **Destino:** Ciudad al cual llega el vuelo.

- **Fecha:** Día programado para el vuelo.
- **Hora de salida:** Hora exacta de despegue del vuelo.
- **Nombre de la aeronave:** Identificación del avión asignado.

Esta funcionalidad está diseñada para facilitar la consulta de opciones de vuelo disponibles, proporcionando una visión clara y organizada al

Diagrama de interacción



Funcionalidad #2: Comprar tickets para un vuelo por destino y fecha.

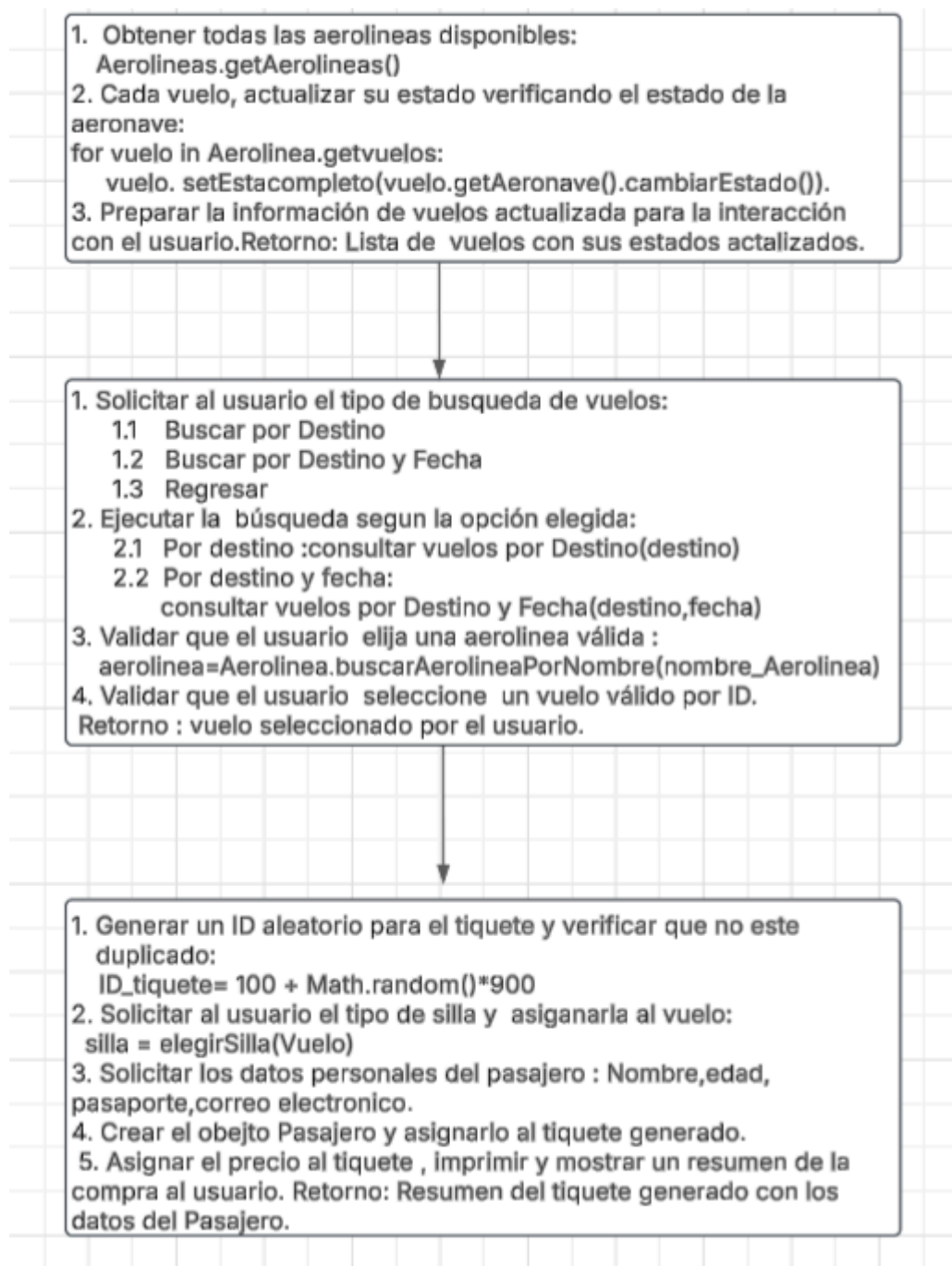
Descripción: En esta funcionalidad le brindará al usuario la opción de comprar un ticket, este podrá elegir si buscar un ticket basado en el destino al cual desea viajar o proporcionando el destino y la fecha en la que quiere realizar su viaje. Según la información ingresada por el usuario, se mostrará una lista de los viajes que cumplen con sus criterios de búsqueda junto con la información relevante de cada uno, este deberá ingresar el nombre de

la aerolínea por la cual desea viajar seguido del ID del vuelo de su elección, también deberá especificar el tipo de silla que desea (económica o ejecutiva) y según el tipo la ubicación, si es ejecutiva venta o pasillo o de ser económica ventana, central o pasillo. Luego se le pedirá ingresar sus datos personales para proceder con la creación del ticket de vuelo.

Cada ticket se imprimirá con los siguientes detalles :

- **Número del ticket:** Único para cada pasaje.
- **Nombre del pasajero:** Corresponde al primer nombre y primer apellido del pasajero.
- **Fecha:** Corresponde a la fecha en que se realizará el vuelo.
- **Número del vuelo :** Este es el ID del vuelo el cual es único de cada uno.
- **Número de la silla y la ubicación :** Hace referencia al número de asiento que se le asignó para dicho vuelo junto con la ubicación.
- **Origen :** Lugar donde saldrá el vuelo.**Destino :** Lugar al que viajará.
- **Precio total:** Valor final que tendrá el pasaje teniendo en cuenta la clase del asiento y la distancia del vuelo.

Diagrama de interacción



Funcionalidad #3: Agregar alojamiento en el destino del vuelo comprado

Descripción

Esta funcionalidad permitirá agregar un alojamiento a un ticket de vuelo ya comprado y registrado a nombre de un usuario, para ello se le preguntará el id del ticket que se generó al momento de comprar el vuelo, según esta información se le mostrará al usuario los alojamientos disponibles para el lugar del destino marcado en el ticket, se le solicitará ingresar el alojamiento que desea incluir en su ticket y los días que se quedará en el alojamiento. Finalmente se actualizará el precio del ticket, es decir al valor generado inicialmente se le sumará el valor de la estadía para dicho alojamiento.

El ticket con su alojamiento agregado se imprimirán en un ticket, especificando la información ahí consignada de la siguiente manera :

Ticket No: Número del ticket único por pasajero

Nombre Pasajero: Corresponde al primer nombre y primer apellido del pasajero.

Fecha: Corresponde a la fecha en que se realizará el vuelo.

Vuelo: Este es el ID del vuelo el cual es único de cada uno.

Silla: Hace referencia al número de asiento que se le asignó para dicho vuelo.

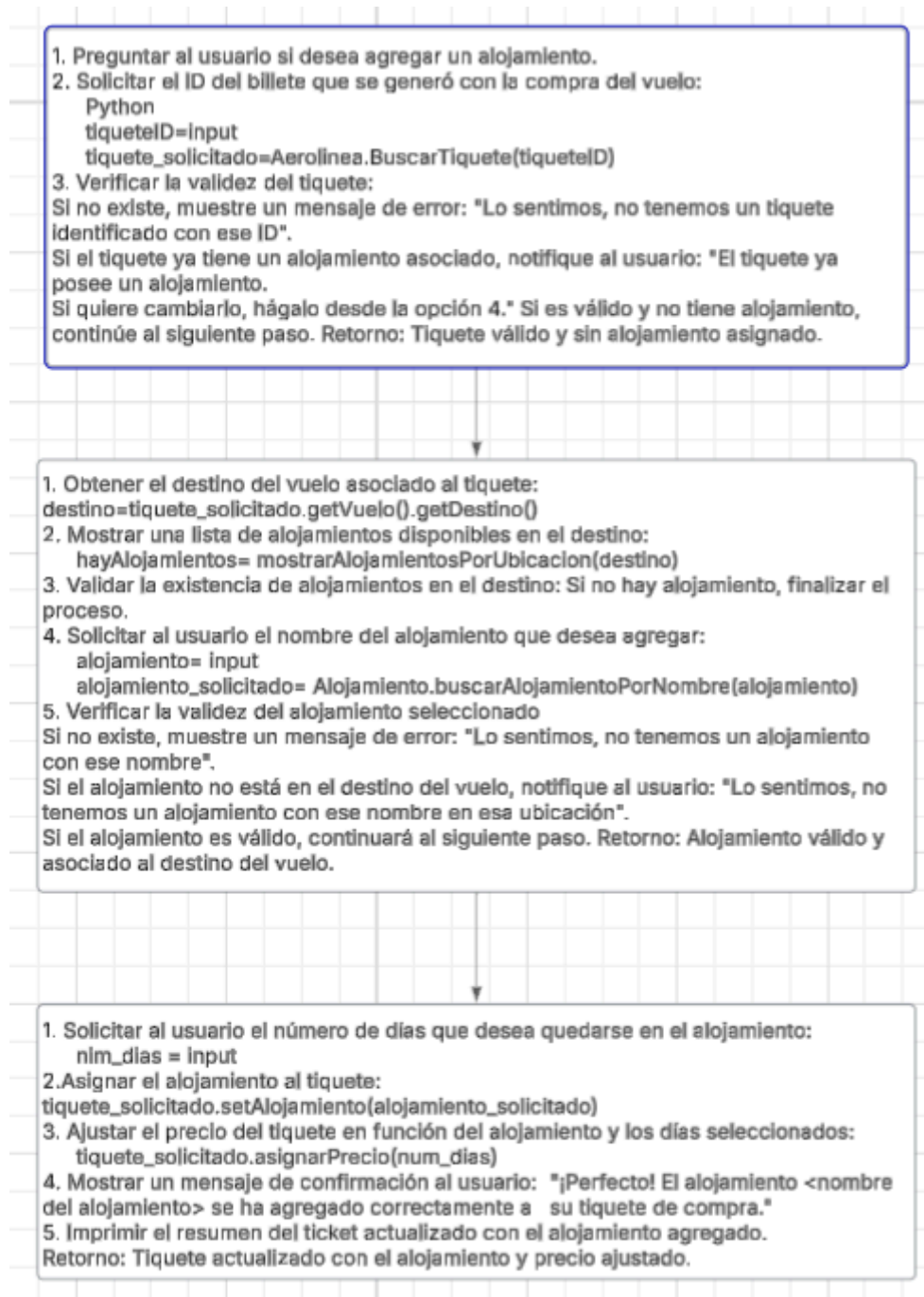
Origen: Lugar donde saldrá el vuelo.

Destino: Lugar al que viajará.

Alojamiento: Este es el que eligió según su destino.

Precio Total: Precio del vuelo más alojamiento.

Diagrama de interacción



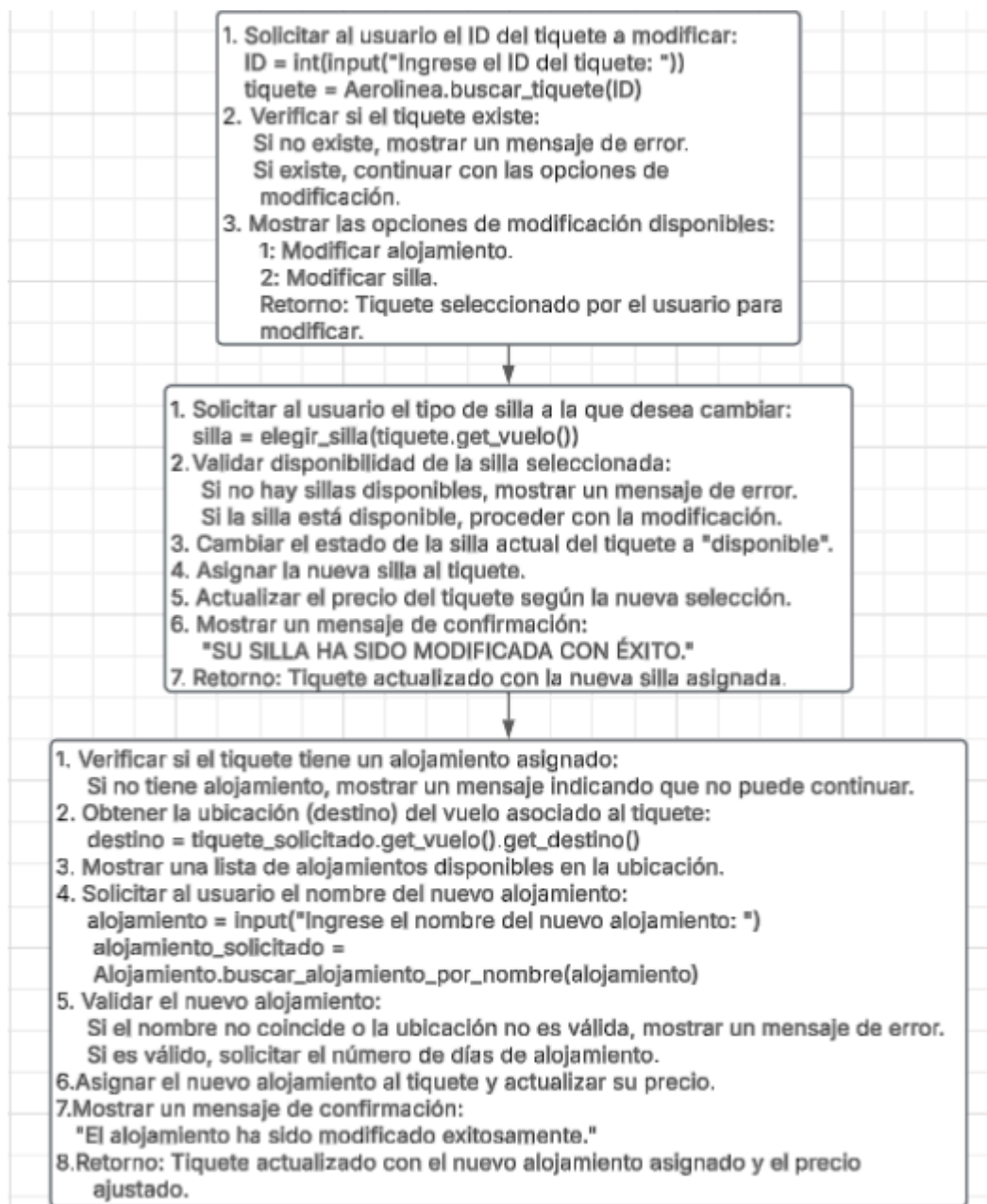
Funcionalidad #4: Modificar tiquete comprado

Descripción

Esta funcionalidad permitirá modificar el alojamiento y la silla de un tiquete previamente comprado (Una modificación a la vez), en el caso de querer modificar la silla se le preguntará a qué tipo de silla desea cambiar (económica o ejecutiva), luego deberá ingresar la ubicación de asiento en que desea ir en caso de no haber del tipo seleccionado se le informará antes de continuar. La información actualizada será agregada en el tiquete para así modificarlo y cambiar su precio.

Si desea modificar el alojamiento se verificará que tenga uno asociado al tiquete, en dado caso que el tiquete no presente un alojamiento asociado se le informará al usuario que podrá agregarle uno en la opción 3 del menú principal (funcionalidad 3). Si el tiquete tiene un alojamiento asociado se podrá cambiar el alojamiento por uno diferente, para ello deberá ingresar el nombre del alojamiento según el destino además de la cantidad de días que desea hospedarse. Para finalizar se realizará un cálculo para ajustar el valor del tiquete según el nuevo alojamiento y la cantidad de días. Este cálculo determinará si se le debe asignar un cargo adicional o restarle al precio total del tiquete.

Diagrama de interacción



Funcionalidad #5: Ver opciones de administrador

Descripción

Esta funcionalidad le mostrará por pantalla al usuario en este caso al administrador un menú de opciones, que tienen como finalidad permitir una mejor gestión del sistema, entre estas opciones podemos destacar:

Listar pasajeros: Para esta opción se tendrá un manejo de datos de todos los pasajeros, según el vuelo y aerolínea al que están asociados para mostrarlos a través de la interfaz .

Agregar Vuelo: En esta opción del menú se le brindará la opción al administrador de agregar un vuelo. Conforme a las aerolíneas ya existentes, se podrá seleccionar una de ellas para agregarle un vuelo con la información pertinente. (ID, precio, origen, destino, distancia, fecha de salida, hora de salida, tipo de aeronave y nombre del avión).

Cancelar un vuelo: Esta opción permitirá la cancelación de un vuelo. Según las aerolíneas y vuelos creados, para ello debe ingresar la aerolínea a la que pertenece dicho vuelo y el ID del vuelo que desea eliminar.

Retirar un avión: Por medio de esta opción el usuario podrá retirar una aeronave. Se le solicitará que ingrese el nombre de la aeronave que desea retirar (previamente se le informo todas las aeronaves existentes por aerolíneas).Luego se realizará la búsqueda de esta y se le cambiara el estado del avión a está “descompuesto” y se eliminarán todos los vuelos asociados a dicha aeronave.

Agregar alojamiento: Ingresado el nombre del alojamiento, la locación, precio por día, número de estrellas podría agregar un alojamiento para que luego pueda ser asociado a cualquier tiquete.

Eliminar alojamiento: Se le mostrará los alojamientos disponibles y se le pedirá el nombre del alojamiento que desea retirar, lo buscará en la lista de alojamientos y será eliminado por completo.

Agregar aerolínea: Se le solicitará un nombre para la aerolínea que desea crear, al hacerlo se añadirá inmediatamente, además se le preguntará si desea agregar vuelos a alguna aerolínea de las previamente ya creadas y podrá agregar solo uno.

Eliminar aerolínea: Se deberá ingresar el nombre de la aerolínea a retirar, y la buscará entre las ya existentes y será eliminada junto con ella, los vuelos que hagan parte de esta.

Agregar aerolínea con vuelos: Brindará la opción de agregar una aerolínea con vuelos (La cantidad que desee) para ello se le preguntará el nombre que le desea asignar a la aerolínea. El sistema verificará si la aerolínea existe, si la aerolínea existe le pedirá que ingrese otro nombre, en caso de que no la aerolínea deberá tener al menos un vuelo así que se le pedirá que ingrese un id (número de tres cifras). Posterior a ello deberá asignar un precio, establecer destino y origen en que operará el vuelo. También deberá agregar una distancia, fecha y hora de salida, además de especificar si la aerolínea será de tipo avión o avioneta, luego pedirá que asigne un nombre para este objeto. Finalmente el sistema mostrará un mensaje haciendo referencia a que la aerolínea ha sido registrada correctamente, luego le preguntará si desea agregar otro vuelo para dicha aerolínea y así hasta que no sea necesario agregar otro vuelo.

Salir del administrador: En esta opción de administrador, se imprimirá: "Gracias por usar nuestras opciones de administrador! y volverá al menú principal.

Diagrama de interacción

Listar Pasajeros

```
+-----+
| 1. Preguntar al usuario si desea listar |
|     pasajeros.                         |
+-----+
| 2. Solicitar el nombre de la aerolínea. |
+-----+
| 3. Verificar la validez de la aerolínea: |
|     • Si no existe, mostrar error.      |
|     • Si existe, continuar.             |
+-----+
| 4. Solicitar el ID del vuelo.            |
+-----+
| 5. Verificar la validez del vuelo:       |
|     • Si no existe, mostrar error.      |
|     • Si existe, mostrar pasajeros.     |
+-----+
```

Agregar Vuelo

```
+-----+
| 1. Preguntar al usuario si desea agregar |
|     un vuelo.                           |
+-----+
| 2. Solicitar el nombre de la aerolínea. |
+-----+
| 3. Verificar la validez de la aerolínea: |
|     • Si no existe, mostrar error.      |
|     • Si existe, continuar.             |
+-----+
| 4. Solicitar datos del vuelo:            |
|     - ID, precio, origen, destino, distancia, |
|       fecha y hora de salida.           |
+-----+
| 5. Seleccionar tipo de aeronave (avión o |
|     avioneta).                          |
+-----+
| 6. Registrar vuelo y mostrar confirmación. |
+-----+
```

Cancelar un vuelo

```
+-----+
| 1. Preguntar al usuario si desea cancelar |
|      un vuelo.                           |
+-----+
| 2. Solicitar el nombre de la aerolínea.   |
+-----+
| 3. Verificar la validez de la aerolínea:  |
|      • Si no existe, mostrar error.       |
|      • Si existe, continuar.             |
+-----+
| 4. Solicitar el ID del vuelo a cancelar.  |
+-----+
| 5. Verificar la validez del vuelo:        |
|      • Si no existe, mostrar error.       |
|      • Si existe, cancelar vuelo.        |
+-----+
```

Retirar Avión

```
+-----+
| 1. Preguntar al usuario si desea retirar |
|      un avión.                           |
+-----+
| 2. Solicitar el nombre de la aeronave.   |
+-----+
| 3. Verificar la validez de la aeronave:  |
|      • Si no existe, mostrar error.       |
|      • Si existe, retirar aeronave y cancelar |
|          vuelo asociado.                 |
+-----+
```

Agregar Alojamiento

```
+-----+
| 1. Preguntar al usuario si desea agregar |
| un alojamiento.                          |
+-----+
| 2. Solicitar datos del alojamiento:      |
| - Nombre, ubicación, precio por día,    |
| número de estrellas.                    |
+-----+
| 3. Crear objeto Alojamiento.             |
+-----+
| 4. Mostrar confirmación de agregado.     |
+-----+
```

Eliminar Alojamiento

```
+-----+
| 1. Preguntar al usuario si desea eliminar |
| un alojamiento.                          |
+-----+
| 2. Mostrar todos los alojamientos disponibles. |
+-----+
| 3. Solicitar nombre del alojamiento.      |
+-----+
| 4. Verificar la validez del alojamiento:  |
| • Si no existe, mostrar error.           |
| • Si existe, eliminarlo.                |
+-----+
```

Agregar Aerolínea

```
+-----+
| 1. Preguntar al usuario si desea agregar |
| una aerolínea.                          |
+-----+
| 2. Solicitar nombre de la aerolínea.     |
+-----+
| 3. Verificar si la aerolínea ya existe.  |
| • Si existe, mostrar error.              |
| • Si no existe, crear aerolínea.         |
+-----+
| 4. Preguntar si desea agregar vuelos.    |
| • Si sí, llamar a agregarNuevoVuelo().  |
| • Si no, finalizar.                     |
+-----+
```

Eliminar Aerolínea

```
+-----+
| 1. Preguntar al usuario si desea agregar |
|      una aerolínea.                      |
+-----+
| 2. Solicitar nombre de la aerolínea.      |
+-----+
| 3. Verificar si la aerolínea ya existe.   |
|      • Si existe, mostrar error.         |
|      • Si no existe, crear aerolínea.    |
+-----+
| 4. Preguntar si desea agregar vuelos.    |
|      • Si sí, llamar a agregarNuevoVuelo(). |
|      • Si no, finalizar.                 |
+-----+
```

Agregar Aerolínea con Vuelos

```
+-----+
| 1. Preguntar al usuario si desea agregar |
|      una aerolínea con vuelos.           |
+-----+
| 2. Solicitar nombre de la aerolínea.      |
+-----+
| 3. Verificar si la aerolínea ya existe.   |
|      • Si existe, mostrar error.         |
|      • Si no existe, continuar.         |
+-----+
| 4. Solicitar datos del vuelo:             |
|      - ID, precio, origen, destino, distancia, |
|      fecha y hora de salida.             |
+-----+
| 5. Seleccionar tipo de aeronave (avión o  |
|      avioneta).                          |
+-----+
| 6. Preguntar si desea agregar más vuelos. |
|      • Si sí, repetir desde el paso 4.   |
|      • Si no, finalizar.                 |
+-----+
```

5. Manejo de excepciones

- Esta excepción extiende de la clase exception. Será la excepción principal de la cual heredarán todas las demás, sirviendo como la base para manejar distintos tipos de errores dentro del sistema.

```
class ErrorAplicacion(Exception):  
  
    def __init__(self, mensaje):  
        self.mensaje_error_inicio = "Manejo de errores de la aplicacion:" + mensaje  
        super().__init__(self.mensaje_error_inicio)
```

- Esta excepción heredará de ErrorAplicación y, además de imprimir su propio mensaje, incluirá también el mensaje de su clase padre. De esta manera, al generarse la excepción, se proporcionará información más completa sobre el error, facilitando su identificación y manejo.

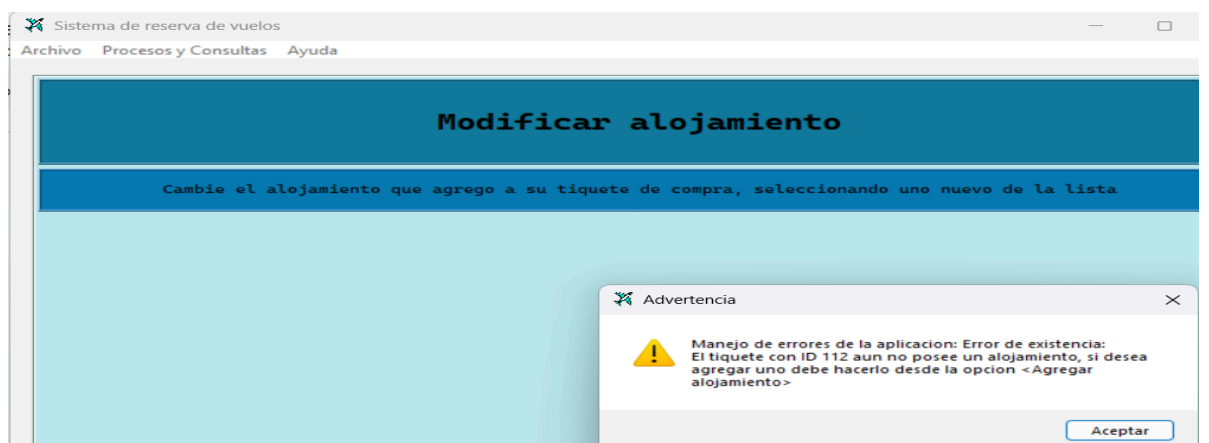
```
from .ErrorAplicacion import ErrorAplicacion  
class ErrorAsignacion(ErrorAplicacion):  
  
    def __init__(self, mensaje):  
        self.mensaje_error_asigancion = f" Error de existencia: {mensaje}"  
        super().__init__(self.mensaje_error_asigancion)
```

- Esta excepción se activará en caso de que un tiquete ya tenga un alojamiento asignado, impidiendo agregar uno nuevo. Si se desea cambiar el alojamiento, deberá utilizarse la función modificarTiquete.

```
class ExcepcionAgregarAlojamiento(ErrorAsignacion):  
  
    def __init__(self, id):  
        self.mensaje_error = f"\nEl tiquete con ID {id} ya posee un alojamiento, si desea cambiarlo debe hacerlo desde la opcion <Modificar tiquete>  
        super().__init__(self.mensaje_error)
```

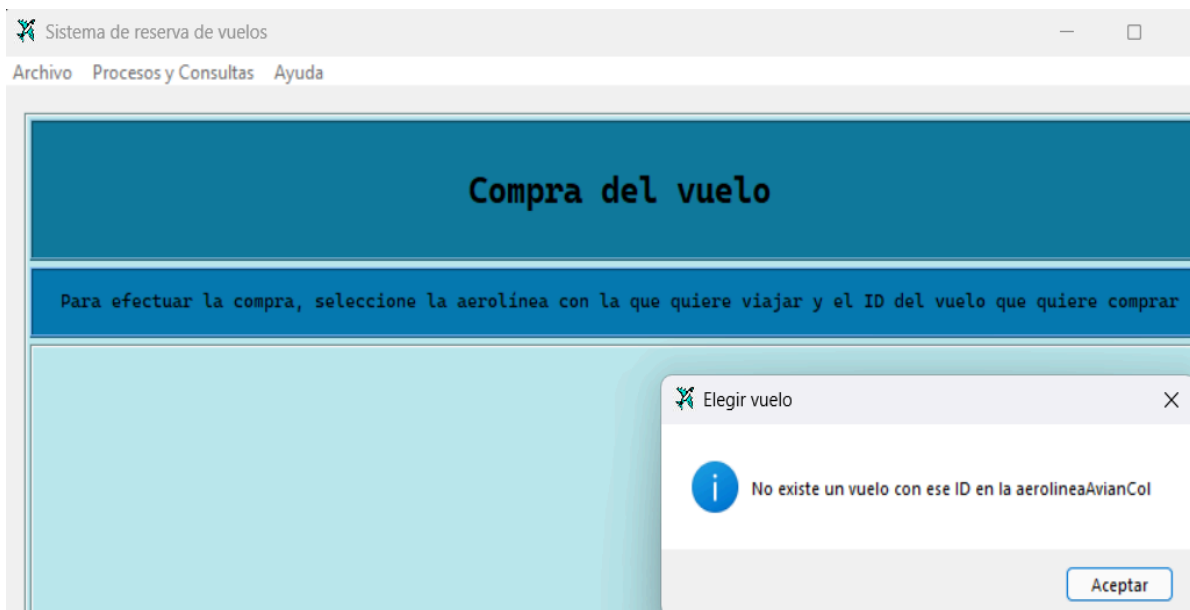
- Esta excepción hereda de ErrorAsignación, que a su vez extiende de ErrorAplicación, la cual deriva de Excepción. En el caso de querer modificar el alojamiento de un tiquete que aún no tiene uno asignado, no podrá seguir la ejecución del código y deberá primero agregar un tiquete.

```
class ExcepcionModificarAlojamiento(ErrorAsignacion):  
  
    def __init__(self, id):  
        self.mensaje_error = f"\nEl tiquete con ID {id} aun no posee un alojamiento, si desea agregar uno debe hacerlo desde la opcion <Agregar alo  
        super().__init__(self.mensaje_error)
```



- Esta excepción entrará en acción si se intenta comprar un ticket para un vuelo que no existe en la aerolínea seleccionada, generando así un error al ingresar el ID y evitando que la compra se complete.

```
class ExcepcionIdVuelo(ErrorAsignacion):  
  
    def __init__(self, id):  
        self.mensaje_error = f"\nNo existe un vuelo con el ID: {id}"  
        super().__init__(self.mensaje_error)
```



- La excepción `IdTicket` se activará si se intenta modificar un ticket que no existe, generando una alerta para que el usuario verifique y modifique el ID del ticket.

```
class ExcepcionIdTicket(ErrorAsignacion):  
  
    def __init__(self, id):  
        self.mensaje_error = f"\nNo existe un ticket con el ID: {id}"  
        super().__init__(self.mensaje_error)
```




Ejemplo de implementación de excepción Id Tiquete:

```
# RECIBE UN VUELO COMO PARAMETRO Y SE ENCARGA DE GENERAR UN TIQUETE CON UN ID ALEATORIO, QUE ES RETORNADO AL FINAL DEL METODO
@staticmethod
def generarTiquete(vuelo):
    ID_tiquete = 100 + random.random() * 900 # DEVUELVE UN NUMERO ALEATORIO DE 3 CIFRAS
    try:
        while Aerolinea.BuscarTiquete(int(ID_tiquete)) is not None:
            ID_tiquete = 100 + random.random() * 900
    except ExcepcionIdTiquete:
        tiquete = Tiquete(int(ID_tiquete), vuelo.getPrecio(), vuelo)
    return tiquete
```

Ejemplo de excepciones ID tiquete, agregar alojamiento y modificar alojamiento.

```
@staticmethod
def buscarTiqueteYAlojamiento(id,numero):
    tiqueteID = int(id)
    tiquete_solicitado = Aerolinea.BuscarTiquete(tiqueteID)

    if tiquete_solicitado == None:
        raise ExcepcionIdTiquete(tiqueteID) # SI NO EXISTE EL TIQUETE

    elif tiquete_solicitado.getAlojamiento() != None:
        if numero == 1:
            raise ExcepcionAgregarAlojamiento(tiqueteID) # SI EL TIQUETE YA TIENE UN ALOJAMIENTO ASOCIADO
        else:
            return tiquete_solicitado
    else:
        if numero == 1:
            return tiquete_solicitado # SI HAY ALOJAMIENTOS DISPONIBLES EN EL LUGAR DE DESTINO
        else:
            raise ExcepcionModificarAlojamiento(tiqueteID)
```

Ejemplo implementación excepción error formato.

```
@staticmethod
def isFloat(s):
    try:
        float(s)
        raise ExcepcionEnteroFloat(s)
    except ValueError:
        return

@staticmethod
def listarPasajeros(valor,label):
    aerolineas = Aerolinea.getAerolineas()
    if not valor.isdigit():
        Admin.isFloat(valor)
        raise ExcepcionEnteroString(valor)

    IDBusqueda = int(valor)

    tiquetes = []
    vuelo = None
    for i in aerolineas:
        if i.buscarVueloPorID(i.getVuelos(), IDBusqueda) != None:
            vuelo = i.buscarVueloPorID(i.getVuelos(), IDBusqueda)
            break
    if vuelo ==None:
        raise ExcepcionIdVuelo(IDBusqueda)

    tiquetes = vuelo.getTiquetes()
    label["text"]+="LISTA DE PASAJEROS PARA EL VUELO " + str(IDBusqueda)

    if len(tiquetes) == 0:
        label["text"]+="\nEl vuelo aun no tiene pasajeros asociados \n"
    else:
        Admin.mostrarTablaDePasajeros(tiquetes,label)
```

6. FieldFrame

Constructor (`__init__`):

El método inicializa el formulario y recibe varios parámetros para personalizar el diseño y comportamiento:

- **Parámetros:**
 - `parent`: Ventana o contenedor padre.
 - `tituloCriterios`: Título para la columna de criterios.
 - `criterios`: Lista de etiquetas que identifican cada campo de entrada.
 - `tituloValores`: Título para la columna de valores.
 - `valoresIniciales`: Valores por defecto de las entradas.
 - `habilitado`: Lista booleana que indica si cada entrada está habilitada o deshabilitada.
 - `tipo_esperado`: Lista con el tipo de dato esperado ("int" o "string").
 - **Estructura de datos internas:**
 - `self.entradas`: Diccionario que asocia cada criterio con su campo de entrada (Entry).
 - `self.valor_entradas`: Lista que almacena los valores ingresados por el usuario.
 - **Diseño visual:**
 - Configura el marco con bordes y color de fondo.
 - Genera etiquetas y campos de entrada de forma dinámica según los criterios recibidos.
 - Aplica validaciones específicas (ej.: limitar la longitud si el criterio empieza con _).
-

Validación y Comportamiento del Formulario:

- Se añade una función interna `validar_longitud()` que limita la longitud de ciertos campos a 3 caracteres.
 - Si se pasan valores iniciales, estos se colocan automáticamente en los campos correspondientes.
 - Si `habilitado` es proporcionado, desactiva o activa campos según la lista booleana.
-

Botones Interactivos:

- Botón Aceptar: Llama a `aceptar()` y realiza validaciones.
 - Botón Borrar: Llama a `borrarValores()` y limpia todos los campos.
 - Ambos botones tienen efectos visuales al pasar el cursor (`inBoton()` y `outBoton()`).
-

Métodos de Validación:

- **aceptar():**
 - Captura los valores ingresados.
 - Lanza una excepción personalizada (`ExcepcionEntradasVacias`) si algún campo está vacío.
 - Valida el tipo de dato esperado:
 - Si se espera un `int`, verifica si el valor ingresado es numérico.
 - Si se espera un `string`, se asegura de que el valor no sea numérico.
 - Si la validación es exitosa, limpia los campos.
 - **borrarValores():**
 - Elimina el contenido de todas las entradas.
 - **getValue(criterio):**
 - Devuelve el valor ingresado en el campo asociado al criterio especificado.
-

Manejo de Excepciones:

Lanza errores personalizados en casos como:

- Campos vacíos (`ExcepcionEntradasVacias`).
 - Tipos de datos incorrectos (`ExcepcionEnteroString`, `ExcepcionStringNumero`).
-

Alcance y Funcionalidad en la Interfaz Gráfica

- **Alcance:**
 - Centraliza la creación de formularios reutilizables.
 - Facilita la validación de datos sin escribir código repetitivo.
 - Asegura la interacción adecuada del usuario con restricciones visuales e interactivas.
- **Funcionalidad:**
 - Permite crear formularios personalizados con títulos, valores iniciales y restricciones.
 - Gestiona errores de entrada con retroalimentación visual (ventanas emergentes de advertencia).
 - Ofrece una estructura de datos clara que permite acceder fácilmente a los valores ingresados.

Field Frames Funcionalidad #2: Comprar ticket para un vuelo por destino y fecha.

Ingrese el ID del vuelo que desea comprar.

INFO	VALOR
ID vuelo	
Aceptar	Borrar

```
formulario = FieldFrame(self.ventana_operaciones,"Info",["_ID vuelo"],"valor",None,None,["int"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “Info”, los criterios que se piden son: ID del vuelo, el título del valor es: “Valor”, no hay valores iniciales ni valores inhabilitados y el tipo esperado del ID del vuelo es “int”.

Ingrese los datos del pasajero.

CRITERIOS	DATOS
Nombre	
Edad	
Pasaporte	
E-mail	
Aceptar	Borrar

```
formulario_pasajero= FieldFrame(self.ventana_operaciones,"criterios",["Nombre","Edad","Pasaporte","E-mail"],"dAtos",None,None,["string","int","string","string"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “criterios”, los criterios que se piden son: Nombre, Edad, Pasaporte y E-mail; el título del valor es: “Datos”, no hay valores iniciales ni valores inhabilitados y los tipos esperados de los valores respectivamente son “string”, “int”, “string”, “string”.

CRITERIO	VALOR
Destino	
Aceptar	Borrar

```
formulario_destino=FieldFrame(self.ventana_operaciones,"Criterio",["Destino"],"Valor",None,None,["string"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “Criterio”, el criterio que se pide es: Destino; el título del valor es: “Valor”, no hay valores iniciales ni valores inhabilitados y el tipo esperado del Destino es “string”.

CRITERIO	VALOR
Destino	
Fecha (DD-MM-AAAA)	
Aceptar	Borrar

```
formulario_destino_fecha=FieldFrame(self.ventana_operaciones,"Criterio",["Destino","Fecha (DD-MM-AAAA)"],"Valor",None,None,["string","string"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “Criterio”, los criterios que se piden son: Destino y Fecha(DD-MM-AAAA), el título del valor es: “Valor”, no hay valores iniciales ni valores inhabilitados y los tipos esperados de los valores respectivamente son “string”, “string”.

Field Frames Funcionalidad #3: Agregar alojamiento en el destino del vuelo.

INFO	
ID del tickete	
Aceptar	Borrar

```
formulario = FieldFrame(self.ventana_operaciones,"info",["_ID del tickete"],"",None,None,["int"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “info”, el criterio que se pide es: ID del tickete; no hay título del valor, no hay valores iniciales ni valores inhabilitados y el tipo esperado del ID del tickete es “int”.

REQUISITO	VALOR
Dias de estadía	
Aceptar	Borrar

```
self.ventana_operaciones = FieldFrame(self.frame,"Requisito",["Dias de estadía"],"valor",["1"],None,["int"])
```

Dentro del frame: frame, el título del criterio es: “Requisito”, el criterio que se pide es: días de estadía; el título del valor es: “valor”, el valor inicial de días de estadía es “1”, no hay valores inhabilitados y el tipo esperado de los días de estadía es “int”.

Field Frames Funcionalidad #4: Modificar tickete comprado.

INFO	
ID del tickete	
Aceptar	Borrar

```
formulario = FieldFrame(self.ventana_operaciones,"info",["_ID del tickete"],"",None,None,["int"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “info”, el criterio que se pide es: ID del tiquete; no hay título del valor, no hay valores iniciales ni valores inhabilitados y el tipo esperado del ID del tiquete es “int”.

```
formulario=FieldFrame(self.ventana_operaciones,"Valores",["Clase","Ubicacion"],"Entradas",None,[True,False,False],["string","string"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “Valores”, los criterios que se piden son: clase y ubicación, el título del valor es: “Entradas”, no hay valores iniciales, El valor de la entrada de clase está habilitado pero el de ubicación está inhabilitado y los tipos esperados de los valores respectivamente son “string”, “string”.

Field Frames Funcionalidad #5: Opciones de administrador.

Listar pasajeros.

```
formulario = FieldFrame(self.ventana_operaciones,"info",["ID del vuelo"],"",["900"],None,["int"])
```

Dentro del frame ventana_operaciones, el título del criterio es: “info”, el criterio que se pide es: ID del vuelo; no hay título del valor, el valor inicial de el ID del vuelo es “900”, no hay valores inhabilitados y el tipo esperado del ID del vuelo es “int”.

Agregar vuelo.

DATOS	VALORES
Aerolinea	
ID (3 cifras)	
Precio	
Origen	
Destino	
Distancia (km)	
Fecha de salida (DD-MM-AAAA)	
Hora de salida (24:00)	
Aeronave	
Nombre aeronave	
Aceptar	Borrar

```
self.ventana_operaciones = FieldFrame(self.frame, "Datos", criterios_vuelo, "Valores", valores_def, None, ["string", "int", "int", "string", "string", "int", "string", "string", "string", "string", "string"])
```

Dentro del frame frame, el título del criterio es: “Datos”, los criterios que se piden son: Aerolínea, ID, Precio, Origen, Destino, Distancia, Fecha de salida (DD-MM-AAAA, Hora de salida (24:00), Aeronave, Nombre aeronave; el título del valor es: “Valores”, no hay valores iniciales ni valores inhabilitados y los tipos esperados de los valores respectivamente son: "string", "int", "int", "string", "string", "int", "string", "string", "string", "string".

Cancelar Vuelo.

INFO	VALOR
Nombre de Aerolinea	
ID	
Aceptar	Borrar

```
self.ventana operaciones = FieldFrame(self.frame, "Info", criterios cancelar_vuelo, "Valor", None, None, ["string", "int"])
```

Dentro del frame frame, el título del criterio es: “Info”, los criterios que se piden son: Nombre de aerolínea y ID del vuelo, el título del valor es: “Valor”, no hay valores iniciales, ni valores inhabilitados y los tipos esperados de los valores respectivamente son “string”, “int”.

Retirar Avión.

INFO	
Nombre de Aerolinea	
Nombre aeronave	
Aceptar	Borrar

```
self.ventana_operaciones = FieldFrame(self.frame,"Info",criterios_retirar_avion,"",None,None,["string","string"])
```

Dentro del frame frame, el título del criterio es: “Info”, los criterios que se piden son: Nombre de aerolínea y nombre de aeronave, no hay título del valor, no hay valores iniciales, ni valores inhabilitados y los tipos esperados de los valores respectivamente son “string”, “string”.

Agregar Alojamiento.

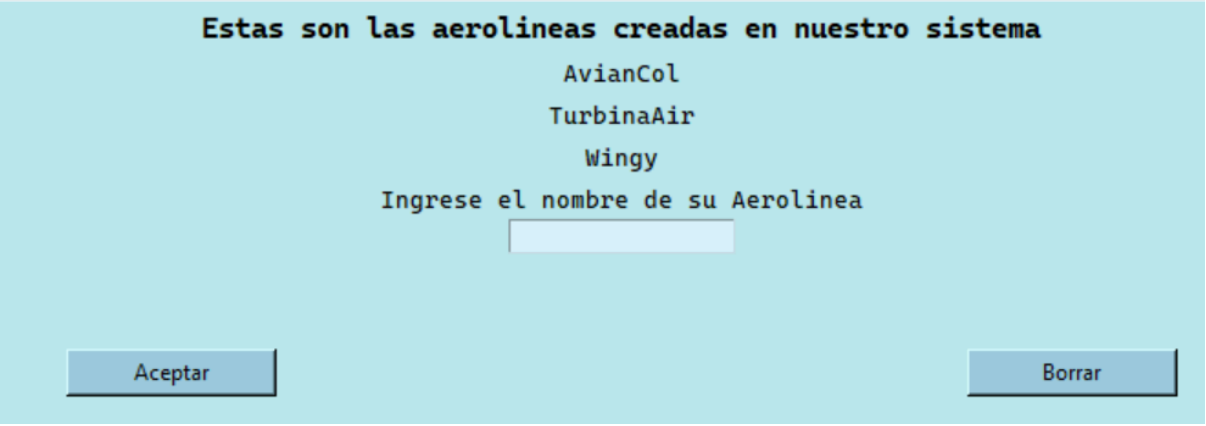
DATOS	VALORES
Nombre Alojamiento	
Locacion	
Precio por dia	
Numero de estrellas (1-5)	
Aceptar	Borrar

```
self.ventana_operaciones = FieldFrame(self.frame,"Datos",criterios_alojamiento,"VALORES",None,None,["string","string","int","int"])
```

Dentro del frame frame, el título del criterio es: “Datos”, los criterios que se piden son: Nombre Alojamiento, Locación, Precio por día y Número de estrellas (1-5), el título del valor

es:”Valores”, no hay valores iniciales, ni valores inhabilitados y los tipos esperados de los valores respectivamente son "string","string","int","int".

Agregar Aerolínea.



```
self.ventana_operaciones = FieldFrame(self.frame,"Datos",criterios_vuelo,"Valores",valores_def,None,["string","int","int","string","string","int","string","string","string","string"])
```

Dentro del frame frame, el título del criterio es: “Datos”, los criterios que se piden son: Aerolínea, ID (3 cifras), Precio, Origen, Destino, Distancia (km), Fecha de salida (DD-MM-AAAA), Hora de salida(24:00), Aeronave y Nombre aeronave; el título del valor es:”Valores”, no hay valores iniciales, ni valores inhabilitados y los tipos esperados de los valores respectivamente son:

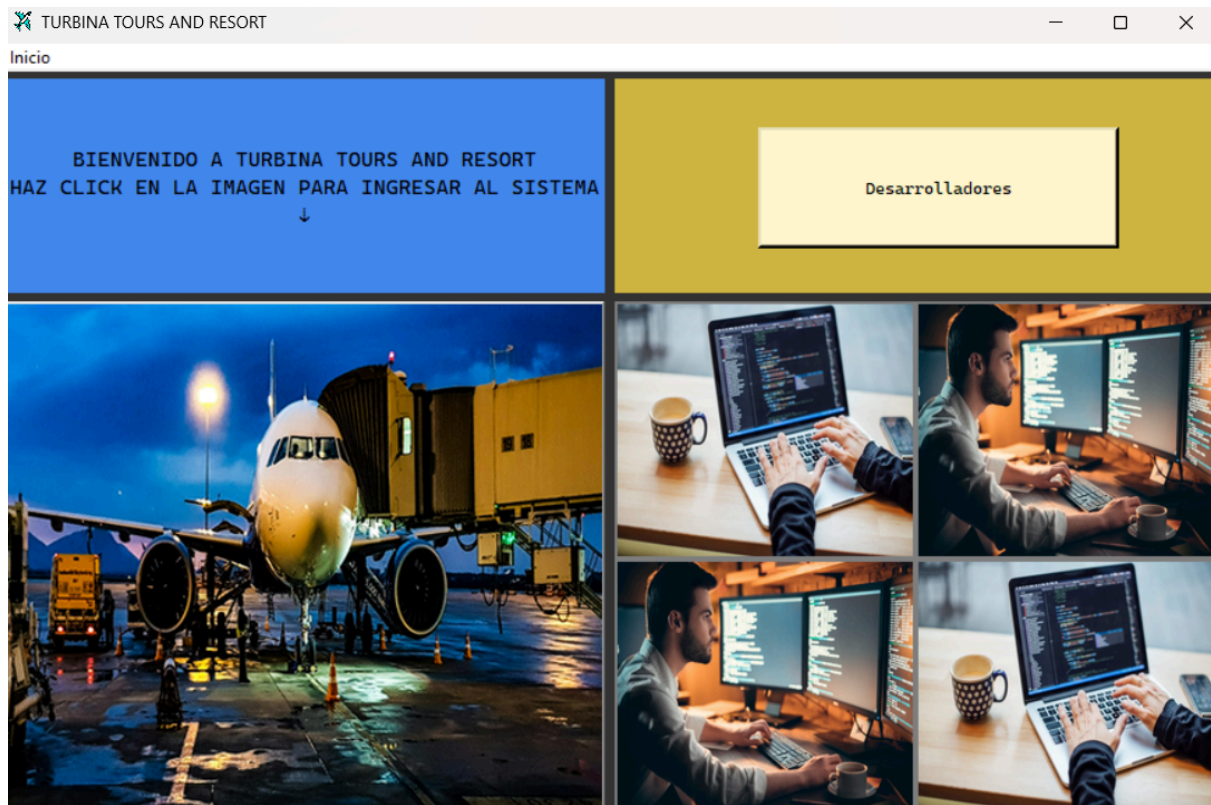
"string","int","int","string","string","int","string","string","string","string".

.

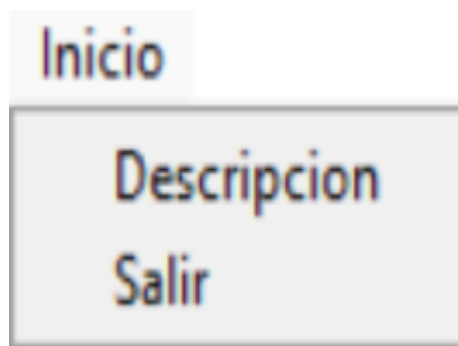
7. Manual de usuario.

El menú principal de la aplicación es de la siguiente manera:

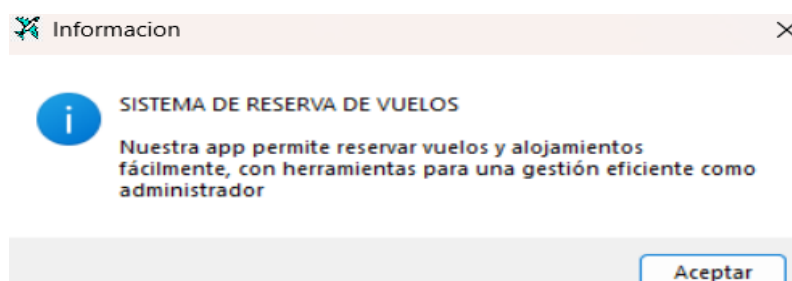
Al ejecutar el programa (star.bat), se abrirá una ventana que le permitirá ingresar al sistema, conocer la hoja de vida de los desarrolladores y acceder al botón de inicio.



El botón de inicio contiene la descripción de la aplicación y permite salir de esta, al hacer click sobre este se le mostrara esta pequeña ventana en la parte superior izquierda:

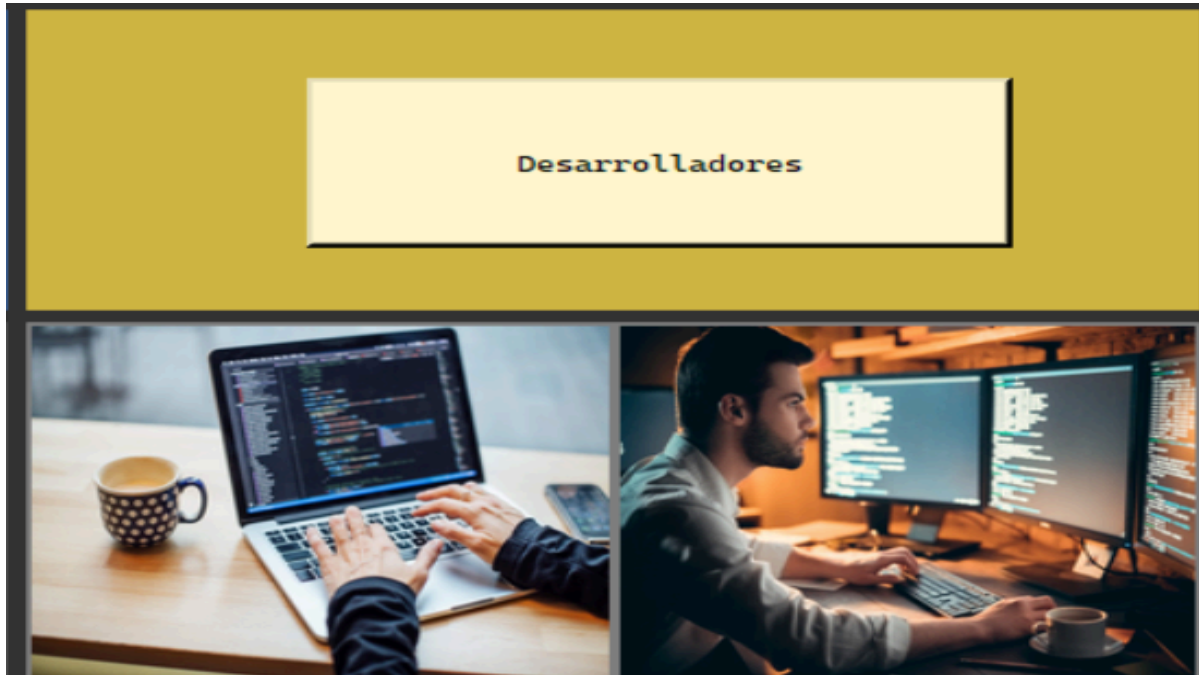


Al darle click al botón descripción le aparecerá una ventana de alerta que le dará una pequeña descripción sobre el programa.



En cambio si das click sobre el botón salir la ventana se destruirá y no te mostrara nada más

Si desea conocer información sobre los desarrolladores, deberá hacer clic en el botón "Desarrolladores". Al hacerlo, se mostrarán imágenes de cada uno junto con sus datos, incluyendo nombre, edad y programa de estudio. Cada vez que se haga clic, se actualizará la información por la de otro desarrollador.



Si desea ingresar a la ventana principal, deberá hacer clic en alguna de las imágenes ubicadas debajo del letrero de bienvenida. Estas imágenes cambiarán cada vez que pase el cursor sobre ellas.

BIENVENIDO A TURBINA TOURS AND RESORT
HAZ CLICK EN LA IMAGEN PARA INGRESAR AL SISTEMA



Al hacer clic en la imagen, el sistema abrirá una nueva ventana con los botones de Archivo, Procesos y Consultas, además de la opción de Ayuda. Y se mostrará el nombre de la aplicación junto con unas indicaciones.

Sistema de reserva de vuelos

Archivo Procesos y Consultas Ayuda

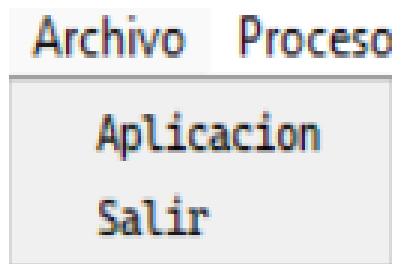
TURBINA TOURS AND RESORT

Sistema para la venta de tiquetes y alojamientos, y modificación por parte de un administrador

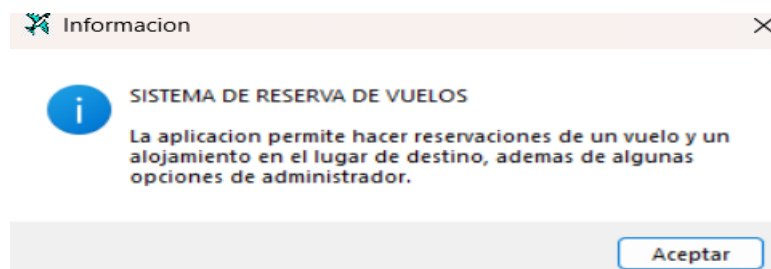
Puedes hacerlo con las acciones dispuestas en el menu <Procesos y consultas>



El botón de Archivo incluye las opciones de Aplicación y Salir.

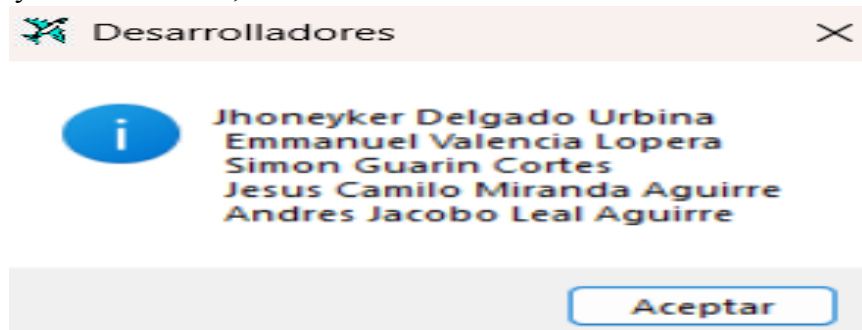


Si das click sobre la opción Aplicación te aparecerá el siguiente mensaje:



Si diste click sobre la opción salir este te mandara directamente a la ventana de inicio mostrada anteriormente.

El botón de Ayuda - Acerca de, contiene información sobre los desarrolladores del programa.



En el botón de procesos y consultas se encuentran todas las opciones o interacciones que usted puede realizar con el sistema.



Sistema de reserva de vuelos

Archivo

Procesos y Consultas

Ayuda

Ver vuelos disponibles por Aerolíneas

Comprar ticket para un vuelo por destino y fecha

Agregar alojamiento en el destino del vuelo

Modificar ticket comprado

Ver opciones de administrador

Si la opción seleccionada en Procesos y Consultas es la #1 (Ver todos los vuelos disponibles por Aerolínea), podrá visualizar una lista de los vuelos disponibles de cada aerolínea con los siguientes datos: ID, Precio, Origen, Destino, Fecha, Hora de Salida y Aeronave.

Vuelos disponibles por aerolínea						
Aquí puede visualizar los vuelos que están disponibles por nuestras aerolíneas						
VUELOS DISPONIBLES DE LA AEROLINEA TURBINAIR						
ID	PRECIO	ORIGEN	DESTINO	FECHA	HORA DE SALIDA	AERONAVE
867	34000	Cali	Villavicencio	15-03-2025	8:00	BusAir-3_A
753	830000	Bogota	New York	09-04-2025	11:00	BusAirPlus_A
234	105000	Bogota	Medellin	02-03-2025	15:00	Ceylan34_A
213	218000	Medellin	Cucuta	03-03-2025	17:00	C-18
941	170000	Villavicencio	Bogota	23-03-2025	22:00	Urraca-20
Siguiente						

Si das click sobre el botón siguiente éste te mostrará otra aerolínea junto con la lista de todos sus vuelos disponibles.

En caso de que haya elegido la opción 2 (Comprar ticket para un vuelo por destino y fecha) se mostrará en pantalla lo siguiente:

Compra de un tiquete

Permite realizar la compra de un tiquete para un vuelo, buscando por destino o por destino y fecha.

Buscar vuelo por:

Destino

Destino y fecha

Si desea buscar un vuelo por destino deberá presionar el botón destino.

Ingresa el destino al que quiere viajar

Buscar vuelo por:

Destino

Destino y fecha

CRITERIO	VALOR
Destino	
Aceptar	Borrar

Luego, el programa le pedirá escribir el destino de su interés, en el espacio en blanco.

En el caso de que no se encuentren vuelos con ese destino, el programa mostrará “Lo sentimos, no tenemos vuelos disponibles para ese destino”.

 Vuelos Disponibles



Lo sentimos, no tenemos vuelos disponibles hacia arauca

Aceptar

Si encuentra uno o más vuelos con el destino de su interés, se los mostrará en pantalla organizados por la aerolínea que lo maneja.

Vuelos disponibles

Lista los vuelos disponibles de acuerdo a los parámetros ingresados

Estos son los vuelos disponibles hacia: cali por nuestras aerolineas

VUELOS DISPONIBLES DE LA AEROLINEA WINGY						
ID	PRECIO	ORIGEN	DESTINO	FECHA	HORA DE SALIDA	AERONAVE
512	217000	Bogota	Cali	03-03-2025	19:30	Ceylan-5
671	173000	Medellin	Cali	08-03-2025	20:10	AirBus-3

Siguiente

Continuar con la compra

Si desea continuar con la compra el sistema le pedirá que seleccione la aerolínea con la que desea viajar.

Compra del vuelo

Para efectuar la compra, seleccione la aerolínea con la que quiere viajar y el ID del vuelo que quiere comprar

Seleccione la aerolinea con la que desea viajar

Luego de seleccionar la aerolínea el sistema le pedirá que ingrese el id del vuelo que desea comprar.

Seleccione la aerolinea con la que desea viajar

Wingy

Ingrese el ID del vuelo que desea comprar.

INFO	VALOR
ID vuelo	671
Aceptar	Borrar

Si el id ingresado es válido , se le pedirá que ingrese su nombre, edad, pasaporte y Email.

Recoge los datos del pasajero al que se le asociará el tiquete de compra

Ingrese los datos del pasajero.

CRITERIOS	DATOS
Nombre	Pedro
Edad	24
Pasaporte	1.12234
E-mail	picapiedra@pedro.com
Aceptar	Borrar

Por lo contrario si el ID no es valido se le mostrara:

Elegir vuelo

 No existe un vuelo con ese ID en la aerolineaAvianCol

Luego de esto el sistema le pedirá que seleccione un tipo de silla(ejecutiva o económica) con su respectiva ubicación (Pasillo,ventana o central).

Resumen de la compra

Enseña un breve resumen de la compra efectuada, recogida en el tiquete

Seleccione el tipo de silla que desea.

VALORES	ENTRADAS
Clase	
Ubicacion	
Aceptar	Borrar

Al finalizar, se mostrará en pantalla que su silla ha sido asignada exitosamente seguido de su tiquete con toda la información de su viaje.

 Elegir ✕

 Su silla ha sido asignada con exito.

Resumen de la compra

Gracias por su compra! Este es su tiquete:

Su compra ha sido exitosa
Gracias por confiar en nostros

Tiquete No.143

Nombre Pasajero: Pedro
Fecha: 08-03-2025
Vuelo: 671
Num Silla: 1 PASILLO
Origen: Medellin
Destino: Cali
Precio Total: 251200

Si desea buscar su vuelo por destino y fecha deberá presionar el botón Destino y fecha, el programa le pedirá ingresar el destino de su interés, seguido de la fecha en la que desea viajar.

Sistema de reserva de vuelos

Archivo Procesos y Consultas Ayuda

Buscar por destino y fecha

Ingresa el destino y la fecha en la que quiere viajar

Buscar vuelo por:

Destino Destino y fecha

CRITERIO	VALOR
Destino	
Fecha (DD-MM-AAAA)	

Aceptar Borrar

Si el programa no encuentra ningún vuelo programado con la información que usted ingresó, se le mostrará

Vuelos Disponibles

Lo sentimos, no tenemos vuelos disponibles hacia Cali en la fecha: 12-03-2025

Aceptar

En el caso de que el programa encuentre uno o más vuelos con sus especificaciones, se le mostrará una lista de los vuelos disponibles separados por cada una de las aerolíneas y se le pedirá que continúe con la compra , al igual que en busca por destino se le pedirá que seleccione la aerolínea con la que desea viajar y el id del vuelo. Si el id ingresado es válido , se le pedirá que ingrese su nombre, edad, pasaporte. Luego de esto el sistema le pedirá que seleccione un tipo de silla(ejecutiva o económica) con su respectiva ubicación (Pasillo, ventana o central). Al finalizar, se mostrará en pantalla que su compra ha sido

exitosa seguido de su tickete con toda la información de su viaje. (Mismo proceso que al buscar por destino)

En caso de haber elegido la opción 3. (Agregar alojamiento en el destino del vuelo).

Al seleccionar esta opción deberá tener presente que para poder utilizar tendrá que tener un tickete previamente. El programa le pedirá el id de su tickete, al ingresarlo se le mostrará una lista con los hoteles disponibles en el destino de su tickete.

Añadir un alojamiento a su compra

Agregue un alojamiento a su tickete de compra, seleccionando uno de la lista

NOMBRE	LOCACION	PRECIO POR DIA	ESTRELLAS
El Buen Alojo	Bogota	50000	3

Usted deberá dar clic sobre el nombre del hotel en el cual le interesaría hospedarse, luego se solicitará ingresar el número de días que desea hospedarse en este alojamiento.(Estará con el numero 1 como predeterminado)

Ingrese los dias que se va a quedar en el alojamiento seleccionado

REQUISITO	VALOR
Dias de estadia	1
Aceptar	Borrar

Posterior a ello se le mostrará un mensaje diciendo que su alojamiento ha sido agregado a su tickete seguido de su tickete de viaje actualizado con la información del hospedaje y su nuevo precio.

Añadir un alojamiento a su compra

Gracias por su compra! Este es su ticket:

Su compra ha sido exitosa
Gracias por confiar en nosotros

Ticket No.730

Nombre Pasajero: Maria Luna
Fecha: 08-03-2025
Vuelo: 900
Silla: 0 - VENTANA
Origen: Medellin
Destino: Bogota
Alojamiento: El Buen Aloj
Precio Total: 741800

En caso de que el destino seleccionado no tenga un alojamiento el sistema le mostrará.

 Agregar alojamiento 



No hay alojamientos en ese destino.

Aceptar

En caso de haber elegido la opción 4 (Modificar ticket comprado), debe tener presente que para hacer uso de esta opción usted deberá tener un ticket previamente .
Para esta opción se le pedirá ingresar el id del ticket que desea modificar, y luego se le mostrará en pantalla lo siguiente:

Modificar Tiquete

Puede modificar los atributos Silla y Alojamiento (si ya tiene uno) de un tiquete comprado previamente

INFO

ID del tiquete

143

Aceptar

Borrar

Al dar click en aceptar se le mostrara:

Modificar Tiquete

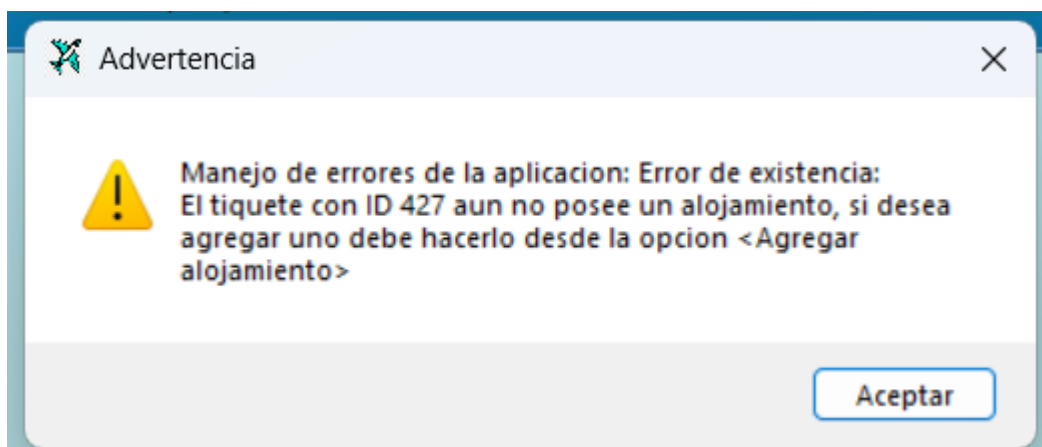
Puede modificar los atributos Silla y Alojamiento (si ya tiene uno) de un tiquete comprado previamente

Que opcion desea modificar de su tiquete

Modificar alojamiento

Modificar silla

Si usted desea modificar el alojamiento debe tener en cuenta que antes debe tener un alojamiento ya enlazado a su tiquete, de no ser asi se le mostrará:



Si este ya tiene uno se le mostrará una lista de los hoteles disponibles en su destino y se le pedirá seleccionar uno para hospedarse(Igual que al agregar un hospedaje en la opción número 3), seguido del número de días que desea estar. Luego se mostrará en pantalla un mensaje diciendo que su alojamiento ha sido modificado exitosamente y podrá ver su tiquete de viaje actualizado con la nueva información del alojamiento y su nuevo precio.

 Modificar alojamiento



Su alojamiento ha sido modificado con éxito.

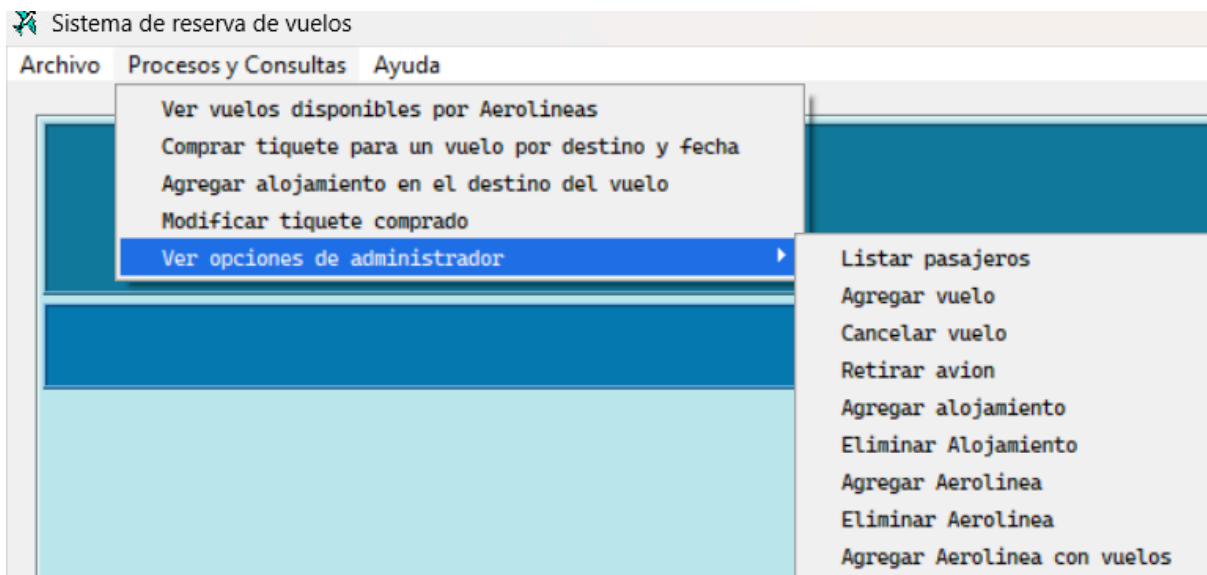
Aceptar

Si usted desea modificar la silla de su tiquete, presione el botón modificar silla y se le mostrarán las siguientes opciones.

Seleccione el tipo de silla que desea.	
VALORES	ENTRADAS
Clase	Economica
Ubicacion	Central
Aceptar	Borrar

En caso de que no exista una silla disponible se le mostrara un mensaje de advertencia, de lo contrario le aparecerá el tiquete modificado.

En caso de haber elegido la opción 5(Ver opciones de administrador) a usted se le mostrará el siguiente menú:



Si usted elige la opción (Listar pasajeros), deberá escribir el número del ID del vuelo de su interés, para facilitar este proceso usted puede ver los vuelos disponibles usando la opción #1 del sistema(Ver todos los vuelos disponibles por Aerolínea).

Permite listar los pasajeros de un vuelo al ingresar su respectivo ID

INFO

ID del vuelo

671

Aceptar

Borrar

Si el vuelo no tiene ningún pasajero se mostrará el mensaje “El vuelo aún no tiene pasajeros asociados.”

Listar pasajeros

Permite listar los pasajeros de un vuelo al ingresar su respectivo ID

INFO

ID del vuelo

Aceptar

Borrar

LISTA DE PASAJEROS PARA EL VUELO 542

El vuelo aun no tiene pasajeros asociados

En el caso de que haya uno o más pasajeros en este vuelo se le mostrará una lista con los pasajeros y la información de cada uno.

Listar pasajeros

Permite listar los pasajeros de un vuelo al ingresar su respectivo ID

INFO

ID del vuelo

Aceptar

Borrar

LISTA DE PASAJEROS PARA EL VUELO 900

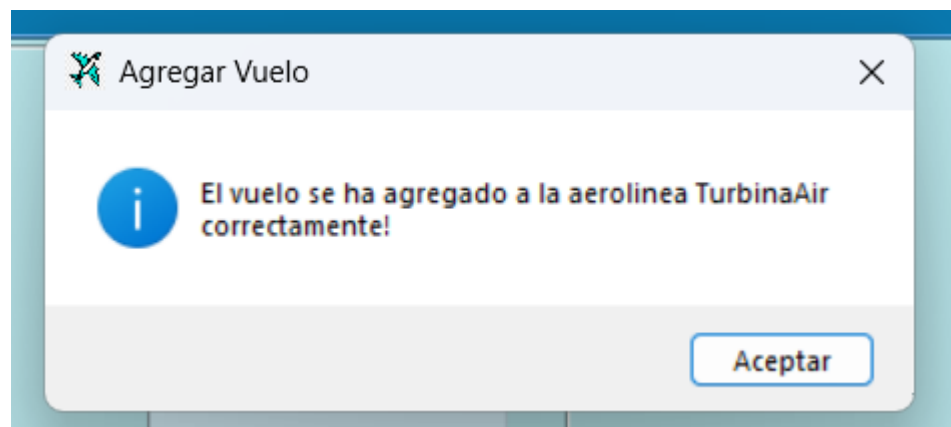
ID	NOMBRE	PASAPORTE	EMAIL
145	Andres Perez	CC109202848	andre92@gmail.co
427	Emmanuel Lopez	TI109382382	emlo@gmail.co
498	Wendy Chacon	CC39283801	wench@gmail.co
794	Fabian Diaz	CC210938303	fabd@gmail.co
730	Maria Luna	TI2183827113	mari@gmail.co

Si usted elige la opción 2 (Agregar vuelo), se le pedirá que seleccione el nombre de la aerolínea a la cual quiere agregarle un vuelo, luego se le pedirán los siguientes datos: Id del vuelo, precio base, origen, destino, distancia en kilometros, fecha de salida, hora de salida, tipo de aeronave y nombre de la aeronave. Cuando haya ingresado todos los datos se mostrará en pantalla “Su vuelo se ha registrado correctamente”.

Agregar un vuelo

Permite agregar un vuelo para ser programado y ofrecido por una de nuestras aerolíneas

DATOS	VALORES
Aerolinea	
ID (3 cifras)	
Precio	
Origen	
Destino	
Distancia (km)	
Fecha de salida (DD-MM-AAAA)	
Hora de salida (24:00)	
Aeronave	
Nombre aeronave	
Aceptar	Borrar



Si usted elige la opción 3 (Cancelar vuelo) Deberá ingresar el nombre de la aerolínea y el id del vuelo que desea retirar, si no se encuentra ningún vuelo con ese id se mostrará el mensaje “No tenemos un vuelo identificado con ese ID”. Si se encuentra un vuelo con este id se eliminará el vuelo y se mostrará “El vuelo se ha eliminado correctamente”.

Cancelar un vuelo

Puedes retirar un vuelo de la lista de vuelos disponibles, escribiendo el nombre del vuelo
(Puede ver el ID de cada vuelo en opcion <Ver vuelos disponibles por Aerolíneas>)

INFO	VALOR
Nombre de Aerolínea	
ID	
Aceptar	Borrar

Si usted elige la opción 4 (Retirar un avión), se le pedirá escribir el nombre de la aeronave que desea retirar , si no se encuentra ninguna aerolínea con ese nombre se mostrará “Lo sentimos, no encontramos una aeronave asociada al nombre que ingresó”. Por el contrario si se encuentra una aerolínea con este nombre, se retirará esta aeronave y sus vuelos asociados, seguido a esto se mostrará “Se ha retirado la aeronave descompuesta y el vuelo asociado a esta”.

Retirar un avion

Retira un avion que esté descompuesto y el vuelo asociado a este
(Puede ver los nombres de las aeronaves en la opcion <Ver vuelos disponibles por Aerolíneas>)

INFO	VALOR
Nombre de Aerolínea	
Nombre aeronave	
Aceptar	Borrar

Si usted elige la opción 5 (Agregar un alojamiento), se le pedirá escribir los siguientes datos: nombre del alojamiento, locación, precio por día y número de estrellas. Si todos los datos están correctos se mostrará el mensaje “Perfecto! El alojamiento hotel se ha agregado a nuestra lista”.

Agregar un alojamiento

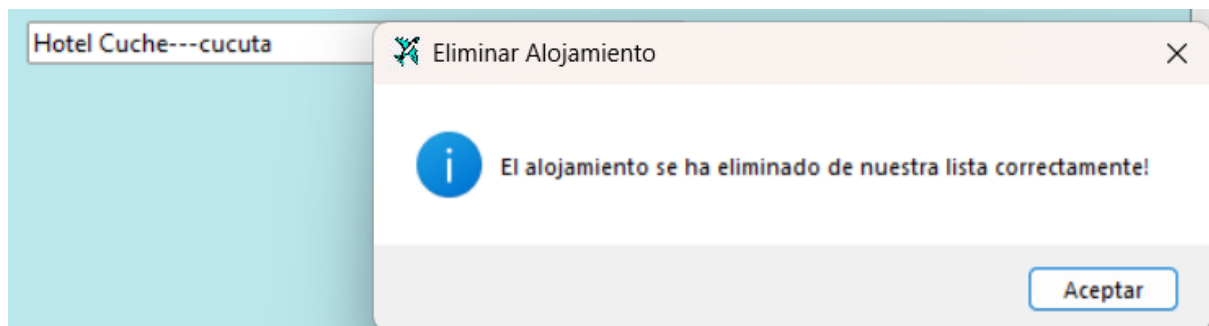
Permite agregar un alojamiento a la lista de alojamientos asociados

DATOS	VALORES
Nombre Alojamiento	
Locacion	
Precio por dia	
Numero de estrellas (1-5)	
Aceptar	Borrar

Si usted elige la opción 6 (Eliminar alojamiento), se le mostrará una lista con cada uno de nuestros alojamientos asociados con sus datos relevantes y se le pedirá ingresar el nombre del hotel que desea eliminar, si este coincide con algún hotel se mostrará “El alojamiento hotel se ha eliminado correctamente”.

Eliminar un alojamiento

Permite eliminar un alojamiento de la lista de alojamientos asociados, seleccionando el nombre del alojamiento que se desea retirar



Si usted elige la opción 7 (Agregar aerolínea), se le pedirá que escriba el nombre que va a llevar la aerolínea que usted desea agregar, si el nombre coincide con alguna aerolínea que ya está registrada, se mostrará “Ya existe una aerolínea con ese nombre”, no se agregará la aerolínea y usted volverá al menú de administrador. En el caso de que no haya ninguna aerolínea con ese nombre, se agregará la aerolínea al sistema y se actualizará la lista.

Agregar aerolínea

Aquí puede agregar aerolíneas a nuestro sistema

Estas son las aerolíneas creadas en nuestro sistema

AvianCol
TurbinaAir
Wingy

Ingrese el nombre de su Aerolinea

Aceptar

Borrar

Si usted elige la opción 8 (Eliminar aerolínea), se le mostrará una lista con las aerolíneas registradas y se le pedirá ingresar el nombre de la aerolínea que desea eliminar . Si no se encuentra ninguna aerolínea con el nombre que usted ingresó, se mostrará “No tenemos una aerolínea con este nombre”. En el caso de que si se encuentre alguna aerolínea con ese nombre, se eliminará la aerolínea junto con los vuelos de esta.

Aquí podrá eliminar una aerolínea con sus vuelos de nuestro sistema

Estas son las aerolíneas existentes en nuestro sistema

AvianCol
TurbinaAir
Wingy
clikcar

Ingrese el nombre de la Aerolinea que desea borrar

Si usted elige la opción 9 (Agregar una aerolínea con vuelos), se le pedirá escribir el nombre de la aerolínea a agregar . Si ya existe una aerolínea con ese nombre, se mostrará “La aerolínea ya existe por favor ingrese otro nombre” y usted deberá ingresar un nombre válido.

 **Agregar Aerolinea** ✕

 **Ya existe una aerolinea con este nombre**

Aceptar

Si el nombre de la aerolínea que usted ingresó no está registrado, deberá crear al menos un vuelo y luego se le pedirán los siguientes datos: Id del vuelo, precio base, origen, destino, distancia en kilómetros, fecha de salida, hora de salida, tipo de aeronave y nombre del avión.

Agregar un vuelo

Permite agregar un vuelo para ser programado y ofrecido por una de nuestras aerolíneas

DATOS	VALORES
Aerolinea	Turbina
ID (3 cifras)	456
Precio	2345
Origen	Bogota
Destino	Cali
Distancia (km)	234
Fecha de salida (DD-MM-AAAA)	12-02-2025
Hora de salida (24:00)	23:00
Aeronave	Avion
Nombre aeronave	ATR 42
Aceptar	Borrar

Cuando haya ingresado todos los datos se mostrará en pantalla “Su vuelo se ha registrado correctamente”.



Agregar Vuelo



El vuelo se ha agregado a la aerolinea Turbina correctamente!

Aceptar

Para guardar los datos en nuestro sistema, deberá elegir la opción inicio-salir en la ventana de inicio.